

PyTorch LocMap

0. 环境

Python 3.8.10

GPU Name: NVIDIA GeForce RTX 4060

Mambaforge

CUDA Version: 12.1

conda environments			
torch_2.0.0	/home/zhangyu/mambaforge/envs/torch_2.0.0	PyTorch Version: 2.4.1+cu121	

实时查看nvidia显卡负载可以输入下面指令：**watch -n 2 -d nvidia-smi**

1. Mambaforge

加载环境: `source ~/mambaforge/bin/activate`

列出环境: `conda env list`

激活环境: `conda activate torch_2.0.0`

退出环境: `conda deactivate`

2. pytorch学习

大纲

第一阶段：PyTorch 核心速成

- Day 1-2: Tensor 玩转数据
 - 学习如何把 GPS 坐标、车速、地图节点数据变成 Tensor。
 - 重点： `device` (CPU/GPU切换), `shape` (维度变换), `autograd` (梯度)。
- Day 3-5: 搭建第一个神经网络

- 任务：写一个简单的 MLP (多层感知机)，输入是车辆的历史轨迹坐标，输出是预测的下一个位置。
- 目的：跑通 Dataset -> Model -> Loss -> Optimizer 的全流程。

第二阶段：SD 地图数据处理 (第2周)

这是你的特色领域。SD 地图通常是 **矢量数据 (Vector)**，由点 (Node) 和 线 (Edge/Link) 组成。

• Day 1-3: GeoPandas 实战

- 学习读取 Shapefile/GeoJSON。
- **坐标系转换 (关键)**：GPS 是 WGS84 (经纬度)，但在定位计算中必须转为投影坐标 (如 UTM)，单位才是“米”。

• Day 4-5: 制作“地图切片”

- 任务：给定一个车辆位置，如何从巨大的路网中“抠”出方圆 100米 内的道路数据，并喂给模型。

第三阶段：模型实战 —— Map Matching (第3-4周)

我们要训练一个模型：**Deep Map Matcher**。

• 输入：

- a. 车辆的一段带噪声的 GPS 轨迹 (Shape: Batch x Sequence x 2)。
- b. 候选道路的几何形状 (Shape: Batch x Num_Roads x Features)。

• 模型：

- 使用 **LSTM** 或 **Transformer** 提取轨迹特征。
- 使用 **Attention** 机制计算轨迹与哪条道路最匹配。

• 输出：车辆到底在哪条路上，以及在路上的具体位置。

知识点

1) Batch Size

loader = DataLoader(dataset, batch_size=32, shuffle=True)

什么是 Batch Size? (核心含义)

Batch Size (批大小) 决定了模型**“一次学多少”**之后才进行一次自我反省 (更新参数)。

想象你在**做题备考** (训练模型)：

- **题库总量 (Dataset Size)**：你有 **1500** 道题。

策略 A：做一道，改一道 (Batch Size = 1)

- **做法：**做第1题 -> 对答案 -> 发现错了 -> 马上调整脑子里的知识点。做第2题 -> 对答案...
- **特点：**
 - **震荡：**如果第1题是错题（标注错了），你的思路就被带偏了。你的学习路线会像无头苍蝇一样乱撞。
 - **太慢：**每次都要停下来改错，效率极低。

策略 B：做完所有题，再一次性改 (Batch Size = 1500)

- **做法：**一口气把 1500 道题全做完 -> 算出总分 -> 计算平均哪里错了 -> 调整知识点。
- **特点：**
 - **内存爆炸：**你的脑子（显存）记不住这 1500 道题的所有细节。
 - **学得慢：**你做完一整套卷子才进步一次，甚至可能陷入死胡同出不来。

策略 C：做一页（32题），改一次 (Batch Size = 32) —— 这就是最佳策略

- **做法：**做 32 道题 -> 算这 32 道的平均误差 -> 调整一次知识点。再做下一组 32 道...
- **特点：**
 - **稳：**因为是取 32 个的平均值，个别错题（噪声）的影响被抵消了，学习方向更准。
 - **快：**GPU 是并行的，它同时计算 32 道题和计算 1 道题的时间几乎一样（GPU 最喜欢批量干活）。

为什么通常写成 32？（玄学与科学）

你会发现深度学习代码里，Batch Size 几乎全是 **16, 32, 64, 128, 256** 这种数字。

原因一：计算机的“强迫症”（二进制与硬件效率）

计算机底层（包括 GPU 的 CUDA 核心）是基于二进制工作的。

- 内存块、线程束（Warps）通常是 **2 的 N 次方** 对齐的。
- 选 **32** 或 **64**，能让 GPU 跑得最顺畅，一点算力都不浪费。如果你设成 33 或 30，GPU 可能要进行填充（Padding），反而变慢。

原因二：黄金平衡点 (Goldilocks Zone)

- **< 16：**太小了，显卡没吃饱，且梯度震荡严重。
- **128：**太大了，普通显卡（比如 8GB 显存）容易爆显存 (**Out of Memory**)；而且研究表明，Batch Size 太大有时会导致模型“死记硬背”，泛化能力变差。
- **32 或 64：**这是经过无数工程师验证过的**“万金油”数字**。在大多数任务上，它既快又稳。

总结对照表

设置	这里的数值	含义	后果
Dataset Size	1500	试卷总题数	数据的总量
Batch Size	32	每次做题数	决定了显存占用和训练速度
Iterations	47 (1500/32)	刷完一遍要改几次错	一个 Epoch 里的步数

一句话建议：

初学者永远先用 32 或 64。

- 如果报错说 “CUDA Out of Memory”（显存不够了），就改成 16。
- 如果显卡很强（比如 A100）且数据很大，可以改成 128 或 256。

day01

第一阶段 Day 1-2：张量 (Tensor) —— 数据的载体

在深度学习中，一切皆 Tensor。

- 一张地图图片是一个 3 维 Tensor (高, 宽, 颜色通道)。
- 一段 10 秒的车辆 GPS 轨迹是一个 2 维 Tensor (时间步, 经纬度)。

你需要掌握三个核心概念：形状 (Shape)、设备 (Device) 和 广播机制 (Broadcasting)。

请在你的 VS Code 或 PyCharm 中创建一个新文件 `day01_tensor.py`，跟随下面的步骤敲代码。

创建 Tensor 与 维度 (Shape)

在智能驾驶中，我们通常不会只处理 “一辆车”，而是处理 “一批数据 (Batch)”。

代码块

```
1  import torch
2
3  # --- 场景模拟 ---
4  # 假设我们有一个 Batch (批次) 的数据，包含 4 辆车。
5  # 每辆车有 3 个特征：[经度 x, 纬度 y, 车速 v]
6  # 数据来自于 CPU (比如刚才用 Pandas 读取的 csv)
7  data_cpu = [
8      [10.0, 20.0, 30.0], # Car 1
9      [11.0, 21.0, 35.0], # Car 2
10     [10.5, 20.5, 0.0], # Car 3 (静止)
11     [12.0, 22.0, 60.0] # Car 4
12 ]
13
14 # 1. 将列表转为 Tensor
15 x = torch.tensor(data_cpu)
```

```
16
17 print(f"数据类型: {x.dtype}") # 默认通常是 float32
18 print(f"数据形状: {x.shape}") # 输出应该是 torch.Size([4, 3])
```

工程师视角: `[4, 3]` 这个形状非常重要。

- `dim=0` (第一个维度) 是 **Batch Size** (4辆车)。
- `dim=1` (第二个维度) 是 **Feature Dim** (3个特征)。以后训练模型时,要是报错 "Dimension mismatch", 90% 都是这里没对齐。

2.GPU 加速 (Device)

你的 RTX 4060 只有把数据搬进去才能发挥作用。

代码块

```
1 # 2. 数据搬运
2 # 检查是否有 GPU, 没有就用 CPU
3 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
4
5 # 将 Tensor 移动到显存中
6 x_gpu = x.to(device)
7
8 print(f"当前设备: {x_gpu.device}")
9
10 # 注意: 存储在 GPU 上的 Tensor 不能直接转 numpy, 必须先转回 CPU
11 # print(x_gpu.numpy()) # 这行会报错
12 print(x_gpu.cpu().numpy()) # 正确做法
```

广播机制 (Broadcasting) —— 定位算法的核心

这是最常用的技巧。场景: 你有一张 SD 地图, 地图上有一个**目标路口**的坐标 `[11.0, 21.0]`。你想一次性计算这 **4辆车** 距离这个路口的距离。

- 车队坐标 shape: `[4, 2]` (取前两列 x,y)
- 路口坐标 shape: `[2]`

PyTorch 会自动把路口坐标“复制”4份, 分别和4辆车做计算。

代码块

```
1 # 取出所有车辆的位置 (所有行, 前两列)
2 # ":" 表示取所有, "0:2" 表示取索引0到1
3 car_positions = x_gpu[:, 0:2] # Shape: [4, 2]
4
5 # 定义路口坐标
6 target_intersection = torch.tensor([11.0, 21.0]).to(device) # Shape: [2]
```

```

7
8 # --- 计算欧式距离 ---
9 # 1. 计算差值 (这里发生了广播: [2] 自动扩展成了 [4, 2] 去进行减法)
10 diff = car_positions - target_intersection
11
12 # 2. 平方
13 sq_diff = diff ** 2
14
15 # 3. 求和 (注意: 要在 dim=1 上求和, 即把 x^2 和 y^2 加起来, 保持 batch 维度不变)
16 dist_sq = torch.sum(sq_diff, dim=1)
17
18 # 4. 开根号
19 distances = torch.sqrt(dist_sq)
20
21 print("---- 距离计算结果 ----")
22 print(f"位置差值:\n{diff}")
23 print(f"每辆车到路口的距离: {distances}")

```

自动求导 (Autograd) —— 神经网络的引擎

PyTorch 与 Numpy 最大的区别在于: PyTorch 的 Tensor 可以记住你是怎么计算它的。

场景: 假设我们要优化车辆的位置 (比如定位有偏差, 我们要通过梯度下降修正位置)。

代码块

```

1 # 创建一个需要求导的参数 (假设这是我们模型预测的初始位置)
2 # requires_grad=True 告诉 PyTorch: 请盯着这个变量的一举一动
3 loc_param = torch.tensor([10.0, 20.0], device=device, requires_grad=True)
4 target = torch.tensor([11.0, 21.0], device=device)
5
6 # 模拟 Loss 计算 (均方误差)
7 loss = torch.sum((loc_param - target) ** 2)
8
9 print(f"当前 Loss: {loss.item()}")
10
11 # 反向传播 (魔法时刻)
12 loss.backward()
13
14 # 查看梯度
15 # 梯度告诉我们: loc_param 里的数据应该怎么变, 才能让 Loss 变小
16 print(f"位置参数的梯度 (Gradient): {loc_param.grad}")
17
18 # 也就是: 2 * (10 - 11) = -2, 2 * (20 - 21) = -2

```

(1) PyTorch 与 Numpy 最大的区别在于：PyTorch 的 Tensor 可以记住你是怎么计算它的，有记忆

第一部分：为什么说 PyTorch 有“记忆”？

1. NumPy vs PyTorch：计算器 vs 录像机

- **NumPy (像个健忘的计算器)**：当你执行 `c = a + b` 时，NumPy 算出 `c` 的值后，就立刻把 `a` 和 `b` 忘掉了。如果你问 `c`：“嘿，你是怎么变出来的？”，`c` 会说：“我不知道，我只知道我现在是几。”
- **PyTorch (像个严谨的录像机)**：当你设置了 `requires_grad=True` (需要求导)，PyTorch 不仅算出结果，还会在后台画一张图，记录下每一个步骤。如果你问 `c`：“你是怎么变出来的？”，`c` 会说：“我是通过一个加法操作 (AddBackward)，由 `a` 和 `b` 变过来的。”

这个记录过程，学名叫“动态计算图” (Dynamic Computational Graph)。

2. 为什么需要“记忆”？（为了秋后算账）

想象你在玩一个射击游戏（就像你的定位模型）：

1. 你开了一枪（前向传播 Forward）：子弹射出去了，偏离了靶心 10 米。
2. 你想修正下一枪（反向传播 Backward）：你需要知道“我刚才枪口是抬高了还是压低了？”以及“我有多少火药？”

NumPy 就像瞎打，打完只知道偏了，不知道为什么偏。PyTorch 记住了你刚才“抬高了手臂”、“扣动了扳机”。所以当它发现偏了 10 米时，它能倒推回去：

- “因为结果偏高了 10 米...”
- “...而你是通过‘抬手臂’这个动作导致高度增加的...”
- “...所以，下一枪请把手臂压低一点点！”

这就是 `loss.backward()` 干的事情：沿着记忆的链路倒回去，找出谁该为误差负责。

(2) 查看梯度中梯度的含义

第二部分：梯度 (Gradient) 到底是什么？

你在代码里看到的 `loc_param.grad`，或者经常听到的“梯度下降”，到底是什么意思？

1. 直观理解：下山的指南针

想象你被蒙住双眼，放在了一座山上（这个山就是 **Loss Function 损失函数**）。你的目标是走到山谷最低点（Loss 最小，即定位最准）。

- **高度 (Loss)**：你站的位置的海拔高度。误差越大，海拔越高。
- **位置 (Parameters)**：你的经纬度坐标（也就是我们要训练的模型参数）。
- **梯度 (Gradient)**：你用脚探一探周围，感觉一下哪个方向最陡峭，最容易往上爬。
 - 梯度永远指向上坡最陡的方向。
 - 梯度的**大小（数值）**代表坡度有多陡。

我们要做的（训练）：既然梯度指向“上坡”，那我们要下山，就必须沿着梯度的反方向走。这就是著名的公式：

$$\text{新的位置} = \text{旧位置} - (\text{学习率} \times \text{梯度})$$

我们在初中/高中学过导数（偏导数就是梯度的分量）：函数 $y = (x - 11)^2$ 求导 $y' = 2 \times (x - 11) \times 1$ （链式法则）

现在，PyTorch 的“记忆”发挥作用了，它自动帮你做了这个求导运算：把 $x = 10$ 代入导数公式：

$$\text{梯度} = 2 \times (10 - 11) = 2 \times (-1) = -2$$

梯度的含义解读：

- **数值是 -2：**
 - **负号**：表示如果你想让 Loss 变大（上坡），你需要减小 x 。反之，如果你想让 Loss 变小（下山），你需要增加 x 。
 - **大小 2**：表示这里的坡度还算陡，你需要迈大步一点。

验证一下：我们要去 11，现在在 10。梯度告诉我们要“增加 x ”（因为梯度是负的，负负得正）。如果你按照梯度的反方向走一步，10 就会变成 10.x，离 11 更近了。这就是模型“学习”的过程！

day02

核心目标：就是把这些手动的操作封装起来，使用 PyTorch 强大的工具箱（`torch.nn`）来构建真正的神经网络，并学会如何把你的 SD 地图数据喂给它。

第一部分：你的第一个神经网络 (The Engine)

我们将抛弃昨天那种手写 `loss.backward()` 和 `w = w - lr * grad` 的原始人方式。

我们要使用 `torch.nn` (Neural Network) 模块。

场景：

假设我们要训练一个极简的轨迹预测器。

- **输入:** 车辆当前的坐标 (x, y) 。
- **输出:** 车辆下一秒预测的坐标 (x', y') 。
- **规律:** 假设车辆在匀速向东北方向行驶，规律大概是 $next = current \times 1.1 + 0.5$ (简单的线性关系)。

定义模型结构 (Class)

在 PyTorch 中，定义模型都要继承 `nn.Module`。这是标准范式。

代码块

```
1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4
5  # 1. 定义一个简单的全连接网络
6  class TrajectoryPredictor(nn.Module):
7      def __init__(self):
8          super().__init__()
9          # 定义层：线性层 (Linear Layer)
10         # 输入特征数 2 (x, y), 输出特征数 2 (next_x, next_y)
11         # 这相当于公式: output = input * Weight + Bias
12         self.layer = nn.Linear(in_features=2, out_features=2)
13
14     def forward(self, x):
15         # 前向传播：数据怎么流过网络
16         return self.layer(x)
17
18 # 2. 实例化模型并移至 GPU
19 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
20 model = TrajectoryPredictor().to(device)
21
22 print("---- 模型结构 ----")
23 print(model)
24 # 查看初始化的随机参数 (Weight 和 Bias)
25 print(f"初始权重: \n{model.layer.weight.data}")
```

准备数据与训练 (Training Loop)

这是深度学习代码最固定的“八股文”：**清空梯度 -> 前向传播 -> 计算Loss -> 反向传播 -> 更新参数**。

代码块

```
1  # --- 模拟数据 ---
2  # 随机生成 100 个点作为输入 (Batch Size = 100)
3  inputs = torch.randn(100, 2).to(device)
4
5  # 生成真实的标签 (Ground Truth)
6  # 假设真实规律是：下个点 = 当前点 * 2 + 1 (我们希望模型学会这个规律)
7  targets = inputs * 2 + 1
8
9  # --- 训练组件 ---
10 # 1. 损失函数：使用 MSE (均方误差)
11 criterion = nn.MSELoss()
12
13 # 2. 优化器：使用 SGD (随机梯度下降)，学习率 lr=0.1
14 # 这一行代替了昨天的 "w = w - lr * grad"
15 optimizer = optim.SGD(model.parameters(), lr=0.1)
16
17 print("\n--- 开始训练 ---")
18 # 训练 1000 轮 (Epochs)
19 for epoch in range(1001):
20     # A. 梯度清零 (非常重要！否则梯度会累加)
21     optimizer.zero_grad()
22
23     # B. 前向传播 (Forward)
24     outputs = model(inputs)
25
26     # C. 计算损失 (Loss)
27     loss = criterion(outputs, targets)
28
29     # D. 反向传播 (Backward)
30     loss.backward()
31
32     # E. 更新参数 (Step)
33     optimizer.step()
34
35     # 每 200 轮打印一次进度
36     if epoch % 200 == 0:
37         print(f"Epoch {epoch}: Loss = {loss.item():.6f}")
38
39 print("\n--- 训练后验证 ---")
40 # 测试一个新数据 [1.0, 1.0]
41 # 真实答案应该是 [3.0, 3.0] (因为 1*2+1=3)
42 test_input = torch.tensor([[1.0, 1.0]]).to(device)
```

```
43 prediction = model(test_input)
44 print(f"输入: [1.0, 1.0]")
45 print(f"模型预测: {prediction.cpu().detach().numpy()}")
46 print(f"真实目标: [3.0, 3.0]")
```

运行这段代码，你应该能看到 Loss 迅速下降接近 0，且预测结果非常接近 `[3.0, 3.0]`。

第二部分：加载 SD 地图数据 (The Fuel)

做智能驾驶定位，最难的不是模型，而是**如何把地图文件 (Shapefile/GeoJSON) 变成 Tensor**。SD 地图通常由一系列的点串 (LineString) 组成。

我们需要用到 `geopandas` 和 `shapely`，然后做坐标归一化（这一点在深度学习中至关重要）。

请在同一个文件或新文件中继续敲：

代码块

```
1 import geopandas as gpd
2 from shapely.geometry import LineString
3 import numpy as np
4
5 print("\n--- 地图数据处理实战 ---")
6
7 # 1. [模拟] 创建一个假的 SD 地图数据
8 # 在实际项目中，你会用 gpd.read_file('map.shp')
9 # 这里我们模拟 3 条道路，坐标是大的墨卡托坐标（单位：米）
10 road_data = {
11     'road_id': [101, 102, 103],
12     'geometry': [
13         LineString([(400000, 3000000), (400100, 3000100)]), # 路段 A
14         LineString([(400100, 3000100), (400200, 3000200)]), # 路段 B
15         LineString([(400200, 3000200), (400300, 3000000)]), # 路段 C
16     ]
17 }
18 gdf_map = gpd.GeoDataFrame(road_data)
19 print("原始地图数据:\n", gdf_map)
20
21 # 2. 提取坐标点并转为 Tensor
22 # 深度学习模型不喜欢巨大的数字（如 400000），会导致梯度爆炸。
23 # 必须进行 [归一化 Normalization]。
24
25 def map_to_tensor(gdf):
26     all_points = []
27
28     for geom in gdf.geometry:
29         # shapely 的 .coords 属性可以拿到 [(x,y), (x,y)...]
```

```

30     points = list(geom.coords)
31     all_points.extend(points)
32
33     # 转为 Numpy 数组
34     np_points = np.array(all_points, dtype=np.float32)
35
36     # --- 关键步骤：归一化 ---
37     # 找到地图的中心点，把所有坐标减去中心点，使其分布在 (0,0) 附近
38     mean_xy = np.mean(np_points, axis=0)
39     std_xy = np.std(np_points, axis=0)
40
41     normalized_points = (np_points - mean_xy) / (std_xy + 1e-6) # 加 1e-6 防止除以0
42
43     # 转为 PyTorch Tensor 并放入 GPU
44     tensor_map = torch.from_numpy(normalized_points).to(device)
45
46     return tensor_map, mean_xy, std_xy
47
48 # 执行转换
49 map_tensor, map_mean, map_std = map_to_tensor(gdf_map)
50
51 print(f"\n转换后的 Tensor (前5个点):\n{map_tensor[:5]}")
52 print(f"Shape: {map_tensor.shape}")
53 print(f"归一化参数 (Mean): {map_mean}")
54
55 # 3. 模拟应用：反归一化 (模型预测出来后，还原回真实地图坐标)
56 # 假设模型输出说车辆在 [0.1, -0.1] (归一化后的坐标)
57 model_output = torch.tensor([[0.1, -0.1]]).to(device)
58
59 # 还原
60 restored_pos = model_output.cpu().numpy() * map_std + map_mean
61 print(f"\n模型输出 (归一化): {model_output.cpu().numpy()}")
62 print(f"还原回地图真实坐标: {restored_pos}")

```

太棒了，进度非常快！

既然你已经理解了“梯度”和“自动求导”，**Day 2 的核心目标**就是把这些手动的操作封装起来，使用 PyTorch 强大的工具箱（`torch.nn`）来构建真正的神经网络，并学会如何把你的 SD 地图数据喂给它。

请创建新文件 `day02_nn_and_map.py`。

第一部分：你的第一个神经网络 (The Engine)

我们将抛弃昨天那种手写 `loss.backward()` 和 `w = w - lr * grad` 的原始人方式。

我们要使用 torch.nn (Neural Network) 模块。

场景：

假设我们要训练一个极简的轨迹预测器。

- **输入:** 车辆当前的坐标 (x, y) 。
- **输出:** 车辆下一秒预测的坐标 (x', y') 。
- **规律:** 假设车辆在匀速向东北方向行驶，规律大概是 $next = current \times 1.1 + 0.5$ （简单的线性关系）。

定义模型结构 (Class)

在 PyTorch 中，定义模型都要继承 `nn.Module`。这是标准范式。

Python

代码块

```
1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4
5  # 1. 定义一个简单的全连接网络
class TrajectoryPredictor(nn.Module):
    def __init__(self):
        super().__init__()
        # 定义层：线性层 (Linear Layer) # 输入特征数 2 (x, y)，输出特征数 2 (next_x, next_y)
        # 这相当于公式: output = input * Weight + Bias
        self.layer = nn.Linear(in_features=2, out_features=2)
6
7
8
9  def forward(self, x): # 前向传播：数据怎么流过网络
    return self.layer(x)
10
11 # 2. 实例化模型并移至 GPU
12 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
13 model = TrajectoryPredictor().to(device)
14
15 print("--- 模型结构 ---")
16 print(model)
17 # 查看初始化的随机参数 (Weight 和 Bias)
18 print(f"初始权重: \n{model.layer.weight.data}")
```

准备数据与训练 (Training Loop)

这是深度学习代码最固定的“八股文”：

清空梯度 -> 前向传播 -> 计算Loss -> 反向传播 -> 更新参数。

Python

代码块

```

1  # --- 模拟数据 ---# 随机生成 100 个点作为输入 (Batch Size = 100)
2  inputs = torch.randn(100, 2).to(device)
3
4  # 生成真实的标签 (Ground Truth)# 假设真实规律是：下个点 = 当前点 * 2 + 1 (我们希望模型
   学会这个规律)
5  targets = inputs * 2 + 1
6
7  # --- 训练组件 ---# 1. 损失函数：使用 MSE (均方误差)
8  criterion = nn.MSELoss()
9
10 # 2. 优化器：使用 SGD (随机梯度下降)，学习率 lr=0.1# 这一行代替了昨天的 "w = w - lr
   * grad"
11 optimizer = optim.SGD(model.parameters(), lr=0.1)
12
13 print("\n--- 开始训练 ---")
14 # 训练 1000 轮 (Epochs)for epoch in range(1001):
15     # A. 梯度清零 (非常重要！否则梯度会累加)
16     optimizer.zero_grad()
17
18     # B. 前向传播 (Forward)
19     outputs = model(inputs)
20
21     # C. 计算损失 (Loss)
22     loss = criterion(outputs, targets)
23
24     # D. 反向传播 (Backward)
25     loss.backward()
26
27     # E. 更新参数 (Step)
28     optimizer.step()
29
30     # 每 200 轮打印一次进度if epoch % 200 == 0:
31         print(f"Epoch {epoch}: Loss = {loss.item():.6f}")
32
33 print("\n--- 训练后验证 ---")
34 # 测试一个新数据 [1.0, 1.0]# 真实答案应该是 [3.0, 3.0] (因为 1*2+1=3)
35 test_input = torch.tensor([[1.0, 1.0]]).to(device)
36 prediction = model(test_input)
37 print(f"输入: [1.0, 1.0]")
38 print(f"模型预测: {prediction.cpu().detach().numpy()}")
39 print(f"真实目标: [3.0, 3.0]")

```

运行这段代码，你应该能看到 Loss 迅速下降接近 0，且预测结果非常接近 `[3.0, 3.0]`。

第二部分：加载 SD 地图数据 (The Fuel)

做智能驾驶定位，最难的不是模型，而是如何把地图文件 (Shapefile/GeoJSON) 变成 Tensor。

SD 地图通常由一系列的点串 (LineString) 组成。

我们需要用到 `geopandas` 和 `shapely`，然后做坐标归一化（这一点在深度学习中至关重要）。

请在同一个文件或新文件中继续敲：

Python

代码块

```
1  import geopandas as gpd
2  from shapely.geometry import LineString
3  import numpy as np
4
5  print("\n--- 地图数据处理实战 ---")
6
7  # 1. [模拟] 创建一个假的 SD 地图数据# 在实际项目中，你会用 gpd.read_file('map.shp')#
   这里我们模拟 3 条道路，坐标是大的墨卡托坐标（单位：米）
8  road_data = {
9      'road_id': [101, 102, 103],
10     'geometry': [
11         LineString([(400000, 3000000), (400100, 3000100)]), # 路段 A
12         LineString([(400100, 3000100), (400200, 3000200)]), # 路段 B
13         LineString([(400200, 3000200), (400300, 3000000)]) # 路段 C
14     ]
15 }
16 gdf_map = gpd.GeoDataFrame(road_data)
17 print("原始地图数据:\n", gdf_map)
18
19 # 2. 提取坐标点并转为 Tensor# 深度学习模型不喜欢巨大的数字（如 400000），会导致梯度爆
   炸。# 必须进行 [归一化 Normalization]。def map_to_tensor(gdf):
20     all_points = []
21
22     for geom in gdf.geometry:
23         # shapely 的 .coords 属性可以拿到 [(x,y), (x,y)...]
24         points = list(geom.coords)
25         all_points.extend(points)
26
27     # 转为 Numpy 数组
28     np_points = np.array(all_points, dtype=np.float32)
29
30     # --- 关键步骤：归一化 ---# 找到地图的中心点，把所有坐标减去中心点，使其分布在
   (0,0) 附近
31     mean_xy = np.mean(np_points, axis=0)
32     std_xy = np.std(np_points, axis=0)
33
```



```

34     normalized_points = (np_points - mean_xy) / (std_xy + 1e-6) # 加 1e-6 防止除以0# 转为 PyTorch Tensor 并放入 GPU
35     tensor_map = torch.from_numpy(normalized_points).to(device)
36
37     return tensor_map, mean_xy, std_xy
38
39 # 执行转换
40 map_tensor, map_mean, map_std = map_to_tensor(gdf_map)
41
42 print(f"\n转换后的 Tensor (前5个点):\n{map_tensor[:5]}")
43 print(f"Shape: {map_tensor.shape}")
44 print(f"归一化参数 (Mean): {map_mean}")
45
46 # 3. 模拟应用：反归一化 (模型预测出来后，还原回真实地图坐标)# 假设模型输出说车辆在 [0.1, -0.1] (归一化后的坐标)
47 model_output = torch.tensor([[0.1, -0.1]]).to(device)
48
49 # 还原
50 restored_pos = model_output.cpu().numpy() * map_std + map_mean
51 print(f"\n模型输出 (归一化): {model_output.cpu().numpy()}")
52 print(f"还原回地图真实坐标: {restored_pos}")

```

Day 2 总结与思考

1. **神经网络**其实就是一个会自动调整参数（权重）的函数。我们用 `optimizer.step()` 来自动完成昨天手动计算梯度的繁琐工作。
2. **数据预处理**是 AI 的生命线。对于地图数据，直接把几十万的坐标值喂给网络是**大忌**，一定要做**归一化 (Normalize)**，把数据压缩到 $[-1, 1]$ 或 $[0, 1]$ 的区间内，模型才能学得快、学得好。

Day 2 课后小任务

尝试修改第一部分的代码：

1. 把 `nn.Linear` 里的 `in_features` 改成 **10** (假设输入过去 10 秒的轨迹点，需把 input 维度改为 `[Batch, 10]`)。
2. 看看会报什么错？(这是新手最常遇到的 Dimension Mismatch 错误)。
3. 尝试修复它 (你需要修改输入数据的形状，或者修改 Linear 层的定义)。

day03

构建专属数据集 (Custom Dataset)：

1. **Dataset (数据集)**：负责**存储或索引**数据（它像一个图书管理员，知道每一本书在哪）。
2. **DataLoader (数据加载器)**：负责**打包**数据（它像一个搬运工，一次搬一箱书给模型）。

场景设定：车道分类 (Lane Classification)

为了配合你的智能驾驶主题，我们今天的任务是：

- **输入 (Inputs):** 车辆的 GPS 坐标 (x, y) 。
- **任务:** 判断这辆车在 **左车道 (Lane 0)** 还是 **右车道 (Lane 1)**。
- **真值 (Targets):** 0 或 1（这是一个**分类**问题）。

请新建文件 `day03_dataset.py`。

第一步：定义 Dataset 类 (核心)

PyTorch 规定，所有的自定义数据集都必须继承 `torch.utils.data.Dataset`，并且必须实现三个“魔法方法”：

1. `__init__`: 初始化（比如读取文件列表）。
2. `__len__`: 告诉 PyTorch 数据总共有多少条。
3. `__getitem__`: **最重要的部分**。给定一个索引 `idx`，你得要把第 `idx` 条数据拿出来，处理好，变成 Tensor 返回去。

代码块

```
1  import torch
2  from torch.utils.data import Dataset, DataLoader
3  import numpy as np
4
5  # 1. 定义我们自己的数据集类
6  class LaneDataset(Dataset):
7      def __init__(self, num_samples=1000):
8          # --- 在这里模拟读取硬盘上的数据 ---
9          # 实际项目中，这里通常是: self.data = pandas.read_csv('gps_logs.csv')
10
11          # 模拟生成 1000 个样本
12          # 假设左车道(Lane 0)的 x 在 0 附近，右车道(Lane 1)的 x 在 10 附近
13          self.num_samples = num_samples
14
15          # 生成 GPS 坐标 (Inputs)
16          # 500辆车在左边，500辆车在右边
17          left_lane_x = np.random.normal(0, 1, size=(num_samples//2, 1))
18          right_lane_x = np.random.normal(10, 1, size=(num_samples//2, 1))
19
20          # y 坐标随机分布（路上前后都有车）
21          y_coords = np.random.uniform(0, 100, size=(num_samples, 1))
22
23          # 合并 x 坐标
24          x_coords = np.vstack((left_lane_x, right_lane_x))
```

```

25
26     # 最终输入数据: Shape [1000, 2] -> (x, y)
27     self.inputs = np.hstack((x_coors, y_coors)).astype(np.float32)
28
29     # 生成标签 (Targets)
30     # 前500个是 0 (左车道), 后500个是 1 (右车道)
31     # 注意: 分类任务的标签通常用 long (整数) 类型
32     self.labels = np.array([0] * (num_samples//2) + [1] *
123 (num_samples//2), dtype=np.int64)
33
34     def __len__(self):
35         # 告诉 PyTorch 数据集有多长
36         return self.num_samples
37
38     def __getitem__(self, idx):
39         # --- 核心: 拿取第 idx 条数据 ---
40
41         # 1. 拿到原始数据
42         x_data = self.inputs[idx]
43         y_label = self.labels[idx]
44
45         # 2. 这里的 x_data 是 numpy, 为了给模型训练, 必须转为 Tensor
46         # 注意: 在这里转 Tensor 比在 __init__ 里转更省内存
47         sample = torch.from_numpy(x_data)
48         label = torch.tensor(y_label) # 标量转 Tensor
49
50         return sample, label
51
52     # 实例化数据集
53     my_dataset = LaneDataset(num_samples=1000)
54
55     # 测试一下能不能拿到数据
56     print(f"数据总量: {len(my_dataset)}")
57     first_data, first_label = my_dataset[0]
58     print(f"第0条数据: 输入={first_data}, 标签={first_label}")

```

太棒了, 进入 **Day 3: 构建专属数据集 (Custom Dataset)** —— 兵马未动, 粮草先行。

前两天我们要么用 `torch.randn` 生成假数据, 要么是手动写个 `for` 循环处理数据。但在实际工程中, 数据可能有几十万条 (比如一整天的行车记录), 塞不进内存, 也不能手动一个个喂给模型。

今天我们要学习 PyTorch 数据管道的两个**核心护法**:

1. **Dataset (数据集)**: 负责**存储或索引**数据 (它像一个图书管理员, 知道每一本书在哪)。
2. **DataLoader (数据加载器)**: 负责**打包**数据 (它像一个搬运工, 一次搬一箱书给模型)。

场景设定：车道分类 (Lane Classification)

为了配合你的智能驾驶主题，我们今天的任务是：

- **输入 (Inputs):** 车辆的 GPS 坐标 (x, y) 。
- **任务:** 判断这辆车在 **左车道 (Lane 0)** 还是 **右车道 (Lane 1)**。
- **真值 (Targets):** 0 或 1（这是一个**分类**问题）。

请新建文件 `day03_dataset.py`。

第一步：定义 Dataset 类 (核心)

PyTorch 规定，所有的自定义数据集都必须继承 `torch.utils.data.Dataset`，并且必须实现三个“魔法方法”：

1. `__init__`: 初始化（比如读取文件列表）。
2. `__len__`: 告诉 PyTorch 数据总共有多少条。
3. `__getitem__`: **最重要的部分**。给定一个索引 `idx`，你得要把第 `idx` 条数据拿出来，处理好，变成 Tensor 返回去。

Python

代码块

```
1  import torch
2  from torch.utils.data import Dataset, DataLoader
3  import numpy as np
4
5  # 1. 定义我们自己的数据集类
class LaneDataset(Dataset):
    def __init__(self, num_samples=1000):
        # --- 在这里模拟读取硬盘上的数据 --- # 实际项目中，这里通常是：
        self.data = pandas.read_csv('gps_logs.csv') # 模拟生成 1000 个样本 # 假设左车道
        # (Lane 0)的 x 在 0 附近，右车道(Lane 1)的 x 在 10 附近
6      self.num_samples = num_samples
7
8      # 生成 GPS 坐标 (Inputs) # 500辆车在左边，500辆车在右边
9      left_lane_x = np.random.normal(0, 1, size=(num_samples//2, 1))
10     right_lane_x = np.random.normal(10, 1, size=(num_samples//2, 1))
11
12     # y 坐标随机分布（路上前后都有车）
13     y_coords = np.random.uniform(0, 100, size=(num_samples, 1))
14
15     # 合并 x 坐标
16     x_coords = np.vstack((left_lane_x, right_lane_x))
17
18     # 最终输入数据: Shape [1000, 2] -> (x, y)
19     self.inputs = np.hstack((x_coords, y_coords)).astype(np.float32)
```

```

20
21         # 生成标签 (Targets)# 前500个是 0 (左车道), 后500个是 1 (右车道)# 注意: 分类
    任务的标签通常用 long (整数) 类型
22         self.labels = np.array([0] * (num_samples//2) + [1] *
    (num_samples//2), dtype=np.int64)
23
24     def __len__(self):# 告诉 PyTorch 数据集有多长return self.num_samples
25
26     def __getitem__(self, idx):# --- 核心: 拿取第 idx 条数据 ---# 1. 拿到原始数据
27         x_data = self.inputs[idx]
28         y_label = self.labels[idx]
29
30         # 2. 这里的 x_data 是 numpy, 为了给模型训练, 必须转为 Tensor# 注意: 在这里转
    Tensor 比在 __init__ 里转更省内存
31         sample = torch.from_numpy(x_data)
32         label = torch.tensor(y_label) # 标量转 Tensorreturn sample, label
33
34 # 实例化数据集
35 my_dataset = LaneDataset(num_samples=1000)
36
37 # 测试一下能不能拿到数据
38 print(f"数据总量: {len(my_dataset)}")
39 first_data, first_label = my_dataset[0]
40 print(f"第0条数据: 输入={first_data}, 标签={first_label}")

```

第二步: 使用 DataLoader (搬运工)

如果不用 `DataLoader`, 你需要写 `for i in range(len(dataset))`, 这效率极低。
`DataLoader` 帮我们做了三件大事:

1. **Batching:** 自动把 1000 条数据切成每份 32 条 (Batch Size)。
2. **Shuffling:** 自动打乱数据顺序 (训练时必须打乱, 否则模型会背答案)。
3. **Parallel:** 可以利用 CPU 多核并行读取 (`num_workers`)。

代码块

```

1 # 2. 定义 DataLoader
2 # batch_size=32: 每次训练喂给模型 32 辆车的数据
3 # shuffle=True: 每次 Epoch 开始时, 把数据顺序打乱 (洗牌)
4 train_loader = DataLoader(dataset=my_dataset, batch_size=32, shuffle=True)
5
6 # 测试 DataLoader
7 # iter() 和 next() 是 Python 获取迭代器第一个内容的写法
8 first_batch_inputs, first_batch_labels = next(iter(train_loader))
9

```

```
10 print(f"\n--- DataLoader 测试 ---")
11 print(f"一个 Batch 的输入形状: {first_batch_inputs.shape}") # 应该是 [32, 2]
12 print(f"一个 Batch 的标签形状: {first_batch_labels.shape}") # 应该是 [32]
```

第三步：完整的训练循环 (分类模型)

现在我们把 Day 2 的模型拿过来，稍微改一下，让它配合 `DataLoader` 工作。因为是分类任务（判断 0 还是 1），我们需要改两个地方：

1. **Loss 函数:** 使用 `CrossEntropyLoss` (交叉熵损失)，这是分类任务的标配。
2. **输出维度:** `out_features=2` (输出两个概率值：是左车道的概率，是右车道的概率)。

代码块

```
1 import torch.nn as nn
2 import torch.optim as optim
3
4 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
5
6 # 1. 定义分类模型
7 class LaneClassifier(nn.Module):
8     def __init__(self):
9         super().__init__()
10        # 输入: (x, y) 2个特征
11        # 输出: 2个类别 (Lane 0 的得分, Lane 1 的得分)
12        self.layer = nn.Linear(2, 2)
13
14    def forward(self, x):
15        return self.layer(x)
16
17 model = LaneClassifier().to(device)
18
19 # 2. 损失函数与优化器
20 # CrossEntropyLoss 内部会自动计算 Softmax, 所以模型输出不需要加 Softmax
21 criterion = nn.CrossEntropyLoss()
22 optimizer = optim.SGD(model.parameters(), lr=0.01)
23
24 print("\n--- 开始训练 (Batch Training) ---")
25
26 # 3. 训练循环
27 for epoch in range(5): # 训练 5 轮
28     total_loss = 0
29
30     # --- 关键变化: 不再是 model(inputs), 而是遍历 loader ---
31     # enumerate 会返回: 批次数号(i), (输入数据, 标签)
32     for i, (batch_inputs, batch_labels) in enumerate(train_loader):
33
```

```

34 # 别忘了把数据搬到 GPU!
35 batch_inputs = batch_inputs.to(device)
36 batch_labels = batch_labels.to(device)
37
38 # A. 清零
39 optimizer.zero_grad()
40
41 # B. 前向
42 outputs = model(batch_inputs)
43
44 # C. 计算 Loss
45 loss = criterion(outputs, batch_labels)
46
47 # D. 反向
48 loss.backward()
49
50 # E. 更新
51 optimizer.step()
52
53 total_loss += loss.item()
54
55 print(f"Epoch {epoch+1}: Average Loss = {total_loss /
56 len(train_loader):.4f}")
57 print("训练完成!")

```

Day 3 知识点解析

- 为什么要 Shuffle (打乱)?** 你的数据集中，前 500 个全是左车道 (0)，后 500 个全是右车道 (1)。如果不打乱，模型在前半段训练时会以为“哇，全世界都是 0”，到了后半段又以为“全世界都是 1”。这会让梯度震荡，没法收敛。 `DataLoader(shuffle=True)` 就像洗牌一样，保证每个 Batch 里既有 0 也有 1，模型学得更稳。
- CrossEntropyLoss 的形状陷阱**
 - `outputs` 形状: `[32, 2]` (32个样本，每个样本2个分类得分)。
 - `labels` 形状: `[32]` (32个整数，代表正确类别的索引)。
 - 注意:** 标签必须是 `long` (整型)，不能是 `float`，否则交叉熵函数会报错。我们在 Dataset 里用 `dtype=np.int64` 保证了这一点。

Day 3 课后作业 (实战挑战)

请修改 `LaneDataset` 类：

- 增加难度:** 现在有 **3 条车道** (Lane 0, Lane 1, Lane 2)。Lane 2 的 x 坐标在 20 附近。
 - 你需要修改 `__init__` 里的数据生成逻辑。

- 你需要修改 `labels` 的生成逻辑。
2. **修改模型**：因为类别变成了 3 个，你需要修改 `nn.Linear` 的 `out_features`。
 3. **观察 Loss**：运行代码，看模型能不能学会区分 3 条车道。

问题：车道分类效果一遍，精度一般

代码块

```
1  数据总量：1500
2  第0条数据：输入=tensor([ 1.6544, 34.7495]), 标签=0
3
4  --- DataLoader 测试 ---
5  一个 Batch 的输入形状：torch.Size([32, 2])
6  一个 Batch 的标签形状：torch.Size([32])
7
8  --- 开始训练 (Batch Training) ---
9  Epoch 1: Loss = 4.0415, Accuracy = 57.13%
10 Epoch 2: Loss = 3.0437, Accuracy = 61.40%
11 Epoch 3: Loss = 1.9036, Accuracy = 70.87%
12 Epoch 4: Loss = 2.2904, Accuracy = 68.60%
13 Epoch 5: Loss = 2.4064, Accuracy = 64.93%
14 训练完成!
```

你有两味药可以治这个病。

方案一：大力出奇迹（增加训练轮数）

既然优化器走得慢，那就让它多走几步。

把 `range(5)` 改成 `range(50)` 或 `range(100)`。

- **预期结果**：准确率最终会升到 99%，但前十几轮可能会很纠结。

方案二：对症下药（数据归一化）—— 推荐！

把 y 这种巨大的噪声压缩到和 x 一样的范围内（比如 0 到 1，或 -1 到 1），模型就能瞬间看清谁才是老大。

请修改 `LaneDataset` 的 `init` 方法，加入**归一化**代码：

Python

代码块

```
1  def __init__(self, num_samples=1500):# ... (前面的代码保持不变) ...# 最终输入数
    据: Shape [1500, 2]
2      self.inputs = np.hstack((x_coords, y_coords)).astype(np.float32)
3
```



```

4      # ===== 新增：归一化 (Normalization) =====# 这一步是提升准确
      率的关键！# 计算均值和标准差
5      mean = self.inputs.mean(axis=0)
6      std = self.inputs.std(axis=0)
7
8      # 执行 Z-Score 归一化：(X - Mean) / Std# 加上 1e-6 是为了防止除以 0
9      self.inputs = (self.inputs - mean) / (std + 1e-6)
10
11     print(f"数据已归一化。Mean: {mean}, Std: {std}")
12     # =====# ... (生成
      labels 的代码保持不变) ...

```

方案三 将SGD改成了Adam优化器

原理总结： 归一化解决了“方向”问题（不再被Y轴噪声误导），优化器升级解决了“速度”问题。两者结合，模型才能起飞。快试试吧！

验证一下

你可以保持 5 个 Epoch 不变，只加上上面的归一化代码，再跑一次。

预期结果：

- **Epoch 1:** Accuracy 直接飙升到 90% 以上。
- **Epoch 5:** Accuracy 应该接近 100%。

为什么？

归一化后， x 和 y 都在 0 附近。 x 因为有明显的聚类（三堆数据），模型一眼就能看出来；而 y 依然是乱的，模型会迅速决定忽略 y （把 w_2 设为 0），专心利用 x 分类。

快去试试！看看加上这几行代码能不能瞬间“起飞”。

day04

模型落地 —— 保存、加载与实际推理 (Inference)

今天我们要完成算法工程师最重要的闭环：

1. **保存 (Save):** 把训练好的“智慧”（权重参数）存成文件。
2. **加载 (Load):** 在另一个程序中读取这个文件。
3. **推理 (Inference):** 模拟真实的自动驾驶场景，对**新来**的数据进行预测。

核心概念：模型文件 .pth

在 PyTorch 中，我们通常保存为 .pth 或 .pt 文件。

我们需要保存两样东西：

1. **模型的参数 (state_dict):** 就是那堆 w 和 b ，这是模型的灵魂。

2. 预处理参数 (Mean/Std): 这是新手最容易漏掉的! 你的模型是在“归一化数据”上训练的, 如果新来的数据不减去同样的 Mean/Std, 模型就听不懂了

第一步: 训练并保存“全套”模型

我们将把昨天的训练代码简化, 重点放在保存环节。

代码块

```
1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  import numpy as np
5  import os
6
7  # --- 1. 快速复现昨天的训练过程 ---
8  # 为了演示, 我们直接生成归一化好的数据, 简化流程
9  num_samples = 1500
10 # 生成三条车道数据 (Lane 0: 0, Lane 1: 10, Lane 2: 20)
11 X = np.vstack((
12     np.random.normal(0, 1, (500, 1)),
13     np.random.normal(10, 1, (500, 1)),
14     np.random.normal(20, 1, (500, 1))
15 ))
16 Y = np.random.uniform(0, 100, (1500, 1)) # 噪声维度
17 inputs_raw = np.hstack((X, Y)).astype(np.float32)
18 labels = torch.tensor([0]*500 + [1]*500 + [2]*500, dtype=torch.long)
19
20 # *** 关键: 计算并记录归一化参数 ***
21 mean = inputs_raw.mean(axis=0)
22 std = inputs_raw.std(axis=0)
23 print(f"[训练阶段] 计算出的 Mean: {mean}, Std: {std}")
24
25 # 归一化
26 inputs_norm = (inputs_raw - mean) / (std + 1e-6)
27 inputs_tensor = torch.from_numpy(inputs_norm)
28
29 # 定义模型
30 model = nn.Linear(2, 3) # 输入2, 输出3分类
31 optimizer = optim.SGD(model.parameters(), lr=0.1)
32 criterion = nn.CrossEntropyLoss()
33
34 print(">>> 开始快速训练...")
35 for i in range(100): # 训练100次确保收敛
36     optimizer.zero_grad()
37     outputs = model(inputs_tensor)
38     loss = criterion(outputs, labels)
39     loss.backward()
```

```

40 optimizer.step()
41
42 print(">>> 训练完成。")
43
44 # --- 2. 保存模型 (Saving) ---
45 # 我们不仅要保存模型权重，还要保存 mean 和 std!
46 # 所以我们创建一个字典来打包所有东西
47 checkpoint = {
48     'model_state_dict': model.state_dict(), # 模型的权重 (w, b)
49     'input_mean': mean,                    # 数据的均值
50     'input_std': std                        # 数据的标准差
51 }
52
53 save_path = 'lane_model_v1.pth'
54 torch.save(checkpoint, save_path)
55 print(f">>> 模型全套参数已保存至: {save_path}")

```

第二步：模拟真实推理 (Inference)

现在，假设你的工作站**重启了**，或者这段代码运行在**自动驾驶域控制器**上。你没有训练数据，只有 `.pth` 文件和**实时传进来的 GPS 信号**。

在同一个文件下方（或者新建一个文件），继续写：

代码块

```

1 print("\n" + "="*30)
2 print("  模拟车辆上路（推理阶段）  ")
3 print("="*30)
4
5 # --- 1. 初始化一个空模型 ---
6 # 必须和训练时的结构完全一致 (Linear 2->3)
7 loaded_model = nn.Linear(2, 3)
8
9 # --- 2. 加载文件 ---
10 if os.path.exists('lane_model_v1.pth'):
11     # 加载字典
12     checkpoint = torch.load('lane_model_v1.pth')
13
14     # A. 恢复模型权重
15     loaded_model.load_state_dict(checkpoint['model_state_dict'])
16
17     # B. 恢复预处理参数
18     saved_mean = checkpoint['input_mean']
19     saved_std = checkpoint['input_std']
20
21 print(">>> 模型加载成功！预处理参数也已就位。")

```

```

22 else:
23     print("!!! 找不到模型文件")
24     exit()
25
26 # --- 3. 切换到推理模式 (非常重要) ---
27 # 这会通知 PyTorch: “我不学习了, 我现在只工作”。
28 # 对于 Linear 层没区别, 但对于 Dropout/BatchNorm 层至关重要。
29 loaded_model.eval()
30
31 # --- 4. 模拟新数据到来 ---
32 # 假设现在来了一辆车, GPS 显示 (x=10.5, y=66.6)
33 # 我们知道这应该是 Lane 1 (中间车道)
34 new_car_raw = np.array([[10.5, 66.6]], dtype=np.float32)
35
36 print(f"\n[新车信号] 原始坐标: {new_car_raw}")
37
38 # --- 5. 预处理 (使用保存的参数) ---
39 # 千万不能用新数据的 mean/std, 必须用训练时的!
40 new_car_norm = (new_car_raw - saved_mean) / (saved_std + 1e-6)
41 new_car_tensor = torch.from_numpy(new_car_norm)
42
43 print(f"[预处理] 归一化后坐标: {new_car_tensor.numpy()}")
44
45 # --- 6. 执行预测 ---
46 # torch.no_grad(): 告诉 PyTorch 别计算梯度了, 省内存, 提速
47 with torch.no_grad():
48     # 前向传播
49     scores = loaded_model(new_car_tensor)
50
51     # 计算概率 (Softmax)
52     probabilities = torch.nn.functional.softmax(scores, dim=1)
53
54     # 获取最大概率的类别
55     predicted_lane = torch.argmax(probabilities, dim=1).item()
56
57 # --- 7. 输出结果 ---
58 lanes = ["左车道 (0)", "中车道 (1)", "右车道 (2)"]
59 print(f"\n>>> 预测结果: 车辆位于 [{lanes[predicted_lane]}]")
60 print(f">>> 置信度: {probabilities.numpy()}")

```

Day 4 关键知识点解析

1. `model.eval()` 和 `torch.no_grad()`

这是推理时的标准起手式。

- `model.eval()`: 冻结某些层的行为（比如 Dropout 层在训练时会随机丢弃神经元，但在推理时必须全开；BatchNorm 在推理时应该用统计好的均值，而不是当前 Batch 的均值）。
- `torch.no_grad()`: 关闭自动求导引擎。推理时我们不需要反向传播，关掉它可以大幅减少显存占用并提高速度。

为什么预处理参数也要保存？

这是一个惨痛的教训。

- **训练时**: $\text{Mean} \approx 10, \text{Std} \approx 8$ 。数据被压缩到了 $[-1, 1]$ 。
- **推理时**: 来了一个新数据 10.5 。
 - 如果你**忘记归一化**: 直接把 10.5 喂给模型。模型会觉得“天哪，这个数字比我见过的最大值还大10倍”，结果一定是错的。
 - 如果你**用新数据的 Mean**: 新数据只有一个点，Mean 就是 10.5 ，相减变成 0 。模型会永远认为这辆车在“平均位置”。
- **正确做法**: 必须用**训练集**的 Mean 和 Std 去处理新数据。

day05

欢迎来到 **Day 5: 进阶之门 —— 处理时序数据 (RNN/LSTM)**。

前四天我们处理的都是**“静态照片”**: 给定一个时刻的 GPS (x, y) ，判断它是哪条车道。

但在自动驾驶中，真正的定位和行为预测是基于“动态视频”**（轨迹）的。

- **静态逻辑**: 车在 $(10, 20)$ $\rightarrow$ 可能是直行，也可能是变道中。
- **动态逻辑**: 车前1秒在 $(8, 20)$ ，现在在 $(10, 20)$ ，预测后1秒在 $(12, 20)$ $\rightarrow$ 这是一个连贯的**轨迹 (Trajectory)**。

今天我们要引入深度学习中处理时间的王者：**LSTM (长短期记忆网络)**。

核心概念：维度的升级

以前你的数据 Shape 是 [Batch, Feature] (比如 [32, 2])。

现在引入时间后，Shape 变成了 3维：

[Batch, Sequence_Length, Feature]

- **Batch**: 32 辆车。
- **Sequence (Seq)**: 每辆车记录过去 **10秒** 的数据。
- **Feature**: 每秒记录 (x, y) 2个坐标。
- **最终 Shape**: `[32, 10, 2]`。

任务目标：驾驶行为识别

我们要训练一个模型，输入一段 **10个点** 的轨迹，判断这辆车在做什么动作：

- **Class 0:** 直行 (Go Straight)
- **Class 1:** 左转 (Turn Left)
- **Class 2:** 右转 (Turn Right)

请新建文件 `day05_lstm.py`。

第一步：训练时序模型

代码块

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import Dataset, DataLoader
5 import numpy as np
6
7 # 定义时序数据集
8 class TrajectoryDataset(Dataset):
9     def __init__(self, num_samples=1000, seq_len=20):
10         self.num_samples = num_samples
11         self.seq_len = seq_len # 序列长度, 比如观测过去 20 个点
12
13     # --- 模拟数据生成 ---
14     # 我们生成三种轨迹, 每种轨迹包含 (x, y) 坐标
15
16     # 1. 直行 (Straight): x 随时间增加, y 只有小波动
17     # Shape: [样本数/3, 序列长, 2]
18     x_straight = np.linspace(0, 10, seq_len) # 0 到 10 均匀分布
19     # 广播机制: 让每个样本的基础 x 都一样
20     X_0 = np.tile(x_straight, (num_samples//3, 1))
21     Y_0 = np.random.normal(0, 0.2, size=X_0.shape) # y 在 0 附近波动
22
23     # 2. 左转 (Left): x 增加, y 向上弯曲 ( $y = 0.1 * x^2$ )
24     X_1 = np.tile(x_straight, (num_samples//3, 1))
25     Y_1 = 0.1 * (X_1 ** 2) + np.random.normal(0, 0.2, size=X_1.shape)
26
27     # 3. 右转 (Right): x 增加, y 向下弯曲 ( $y = -0.1 * x^2$ )
28     X_2 = np.tile(x_straight, (num_samples//3, 1))
29     Y_2 = -0.1 * (X_2 ** 2) + np.random.normal(0, 0.2, size=X_2.shape)
30
31     # --- 合并数据 ---
32     # 现在的 Shape 需要合并成: [1000, 20, 2]
33     # 这里的 stack 稍微有点绕, 只要记得我们需要 (Batch, Seq, Feat)
```

```

35     # 先合并 X 和 Y
36     # data_0 shape: [333, 20, 2]
37     data_0 = np.stack([X_0, Y_0], axis=2)
38     data_1 = np.stack([X_1, Y_1], axis=2)
39     data_2 = np.stack([X_2, Y_2], axis=2)
40
41     # 合并所有样本
42     self.inputs = np.concatenate([data_0, data_1, data_2],
axis=0).astype(np.float32)
43
44     # 生成标签 (0, 1, 2)
45     self.labels = np.array([0]*(num_samples//3) + [1]*(num_samples//3) +
[2]*(num_samples//3), dtype=np.int64)
46
47     # --- 同样要记得归一化! (针对所有点的 x 和 y) ---
48     # 为了简单, 这里只打印 shape 确认一下
49     print(f"数据集构建完成。Shape: {self.inputs.shape}") # 应该是 [1000, 20,
2]
50
51     def __len__(self):
52         return self.num_samples
53
54     def __getitem__(self, idx):
55         # 拿出来直接就是 [20, 2] 的 Tensor
56         return torch.from_numpy(self.inputs[idx]),
torch.tensor(self.labels[idx])
57
58     # 实例化
59     dataset = TrajectoryDataset(num_samples=1500, seq_len=20)
60     loader = DataLoader(dataset, batch_size=32, shuffle=True)
61
62     class TrajectoryClassifier(nn.Module):
63         def __init__(self, input_size=2, hidden_size=64, num_classes=3):
64             super().__init__()
65
66             # --- 定义 LSTM 层 ---
67             # input_size=2 : 输入特征是 x,y
68             # hidden_size=64 : LSTM 内部的记忆容量 (可以理解为提取了 64 个特征)
69             # batch_first=True : 告诉 PyTorch 输入数据的第一个维度是 Batch (最常用)
70             self.lstm = nn.LSTM(input_size=input_size,
hidden_size=hidden_size,
71                                 batch_first=True)
72
73             # --- 定义全连接层 (分类头) ---
74             # 把 LSTM 提取的 64 个特征, 映射到 3 个分类上
75             self.fc = nn.Linear(hidden_size, num_classes)
76
77

```

```

78 def forward(self, x):
79     # x 的形状: [Batch, Seq_Len, Feature] -> [32, 20, 2]
80
81     # 1. 过 LSTM
82     # LSTM 返回两个值:
83     # out: 所有时间步的输出 [32, 20, 64]
84     # (h_n, c_n): 最后一个时间步的记忆状态
85     out, (h_n, c_n) = self.lstm(x)
86
87     # 2. 取“最后一个时间点”的输出
88     # 因为我们看完了整段轨迹, 要在最后时刻做决定
89     # out[:, -1, :] 意思是: 所有样本(:), 最后一个时间步(-1), 所有特征(:)
90     # 形状变成: [32, 64]
91     last_time_step_feature = out[:, -1, :]
92
93     # 3. 过分类层
94     x = self.fc(last_time_step_feature)
95     return x
96
97 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
98 model = TrajectoryClassifier().to(device)
99
100 # 使用 Adam (你已经知道它比 SGD 快了)
101 optimizer = optim.Adam(model.parameters(), lr=0.01)
102 criterion = nn.CrossEntropyLoss()
103
104 print(">>> 开始训练轨迹分类模型...")
105
106 for epoch in range(10): # 10轮就够了, LSTM 学这种简单规律很快
107     total_loss = 0
108     correct = 0
109     total = 0
110
111     for i, (inputs, labels) in enumerate(loader):
112         inputs = inputs.to(device) # Shape: [32, 20, 2]
113         labels = labels.to(device)
114
115         optimizer.zero_grad()
116         outputs = model(inputs) # Forward
117         loss = criterion(outputs, labels)
118         loss.backward()
119         optimizer.step()
120
121         total_loss += loss.item()
122
123     # 计算精度
124     _, predicted = torch.max(outputs, 1)

```



```

125     total += labels.size(0)
126     correct += (predicted == labels).sum().item()
127
128     print(f"Epoch {epoch+1}: Loss={total_loss/len(loader):.4f}, Acc=
{100*correct/total:.2f}%")
129
130     print(">>> 训练完成! ")
131
132     # --- 2. 保存模型 (Saving) ---
133     # 我们不仅要保存模型权重, 还要保存 mean 和 std!
134     # 所以我们创建一个字典来打包所有东西
135     checkpoint = {
136         'model_state_dict': model.state_dict(), # 模型的权重 (w, b)
137     }
138
139     save_path = 'turn_model_v1.pth'
140     torch.save(checkpoint, save_path)
141     print(f">>> 模型全套参数已保存至: {save_path}")
142

```

任务：模拟实时数据流的推理。

假设你的车正在开，传感器每 0.1秒 给你一个坐标。你需要积攒 **20个点** 才能喂给模型。

请写一段伪代码或 Python 代码，实现一个 “**滑动窗口 (Sliding Window)**”：

1. 创建一个长度为 20 的 `buffer` (列表或 deque)。
2. 写一个 `while True` 循环模拟接收新坐标 `(x, y)`。
3. 每次把新坐标加进 `buffer`。如果 `buffer` 满了（超过 20个），就踢掉最老的一个（保持长度 20）。
4. 当 `buffer` 长度刚好等于 20 时，把它转换成 Tensor `[1, 20, 2]`，喂给模型进行预测。

代码块

```

1  import torch
2  import torch.nn as nn
3  import numpy as np
4  from collections import deque # <--- 神器在这里
5  import time
6  import random
7
8  # =====
9  # 1. 模型定义 (必须和训练时一致)
10 # =====
11 class TrajectoryClassifier(nn.Module):
12     def __init__(self, input_size=2, hidden_size=64, num_classes=3):

```

```

13     super().__init__()
14     self.lstm = nn.LSTM(input_size=input_size, hidden_size=hidden_size,
batch_first=True)
15     self.fc = nn.Linear(hidden_size, num_classes)
16
17     def forward(self, x):
18         out, _ = self.lstm(x)
19         last_time_step = out[:, -1, :]
20         x = self.fc(last_time_step)
21         return x
22
23     # =====
24     # 2. 核心类：滑动窗口推断器
25     # =====
26     class RealTimePredictor:
27         def __init__(self, model_path, mean, std):
28             # A. 加载模型
29             self.model = TrajectoryClassifier()
30             # 这里为了演示，如果找不到文件就不加载，实际使用请确保文件存在
31             try:
32                 checkpoint = torch.load(model_path)
33                 self.model.load_state_dict(checkpoint['model_state_dict'])
34                 print(">>> 模型权重加载成功！")
35             except:
36                 print(">>> [警告] 未找到模型文件，使用随机初始化模型演示流程")
37
38             self.model.eval() # 开启推理模式
39
40             # B. 记录归一化参数（必须用训练集的！）
41             self.mean = np.array(mean, dtype=np.float32)
42             self.std = np.array(std, dtype=np.float32)
43
44             # C. 初始化滑动窗口（关键点！）
45             # maxlen=20: 保证列表永远最多只有 20 个元素
46             # 一旦第 21 个进来，第 1 个自动被踢走
47             self.buffer = deque(maxlen=20)
48
49         def process_new_point(self, x, y):
50             # 1. 将新点加入窗口
51             self.buffer.append([x, y])
52
53             # 2. 检查数据够不够 20 个
54             if len(self.buffer) < 20:
55                 print(f"\r[积攒数据] 当前缓冲池大小: {len(self.buffer)}/20", end="",
flush=True)
56                 return None # 数据不够，不预测
57

```

```

58 # 3. 数据够了, 开始推理!
59 # 3.1 转为 Numpy [20, 2]
60 seq_data = np.array(self.buffer, dtype=np.float32)
61
62 # 3.2 归一化 (关键!)
63 seq_norm = (seq_data - self.mean) / (self.std + 1e-6)
64
65 # 3.3 转 Tensor 并增加 Batch 维度
66 # [20, 2] -> [1, 20, 2]
67 input_tensor = torch.from_numpy(seq_norm).unsqueeze(0)
68
69 # 3.4 模型预测
70 with torch.no_grad():
71     scores = self.model(input_tensor)
72     probs = torch.nn.functional.softmax(scores, dim=1)
73     pred_idx = torch.argmax(probs, dim=1).item()
74
75     return pred_idx, probs.numpy()[0]
76
77 # =====
78 # 3. 主程序: 模拟传感器数据流
79 # =====
80 if __name__ == "__main__":
81     # 假设的训练集统计数据 (你需要填入你训练时的真实值)
82     # 这里随便写几个防报错
83     saved_mean = [5.0, 0.0]
84     saved_std = [2.0, 1.0]
85
86     # 实例化推断器
87     predictor = RealTimePredictor('turn_model_v1.pth', saved_mean, saved_std)
88
89     print("\n>>> 开始接收传感器数据流...")
90
91     # 模拟一个左转弯的过程 (X增加, Y平方增加)
92     # 生成 100 个连续的时间点
93     for i in range(100):
94         # --- 模拟传感器产生数据 ---
95         t = i * 0.5
96         sensor_x = t
97         sensor_y = 0.05 * (t ** 2) + random.uniform(-0.1, 0.1) # 加点噪声
98
99         # --- 喂给推断器 ---
100         result = predictor.process_new_point(sensor_x, sensor_y)
101
102         # --- 如果有结果就打印 ---
103         if result is not None:
104             pred_class, conf = result

```

```

105     labels = ["直行", "左转", "右转"]
106
107     # 打印结果 (用 \r 覆盖上一行, 做出动态效果)
108     print(f"\r>>> 时刻 {i}: 坐标({sensor_x:.1f}, {sensor_y:.1f}) | "
109           f"预测: [{labels[pred_class]}] | "
110           f"置信度: 左{conf[1]:.2f} / 直{conf[0]:.2f} / 右{conf[2]:.2f}
111
112     ", end="")
113
114     # 模拟 0.1 秒的传感器间隔
115     time.sleep(0.1)
116
117     print("\n>>> 模拟结束")

```

Day 5 关键点解析

1. 数据的“形状” (Shape): 这是学习 LSTM 最难的一关。
 - Day 4: [Batch, 2] (点)
 - Day 5: [Batch, 20, 2] (线/轨迹)
 - LSTM 内部会自动循环处理这 20 个点, 你不需要自己写 for 循环。
2. batch_first=True: PyTorch 的 LSTM 默认输入形状是 [Seq, Batch, Feat]。这很反人类 (通常我们认为 Batch 在最前)。加上 batch_first=True, 就可以使用我们可以理解的 [Batch, Seq, Feat] 格式。
3. 取最后一个时间步 out[:, -1, :]: LSTM 处理完第 1 个点, 有一个输出; 处理完第 2 个点, 也有一个输出... 既然我们要判断“整段轨迹”是什么, 通常只需要看它“看完所有点后”的最后那个结论。

day06

欢迎来到 Day 6: 终极实战 —— Deep Map Matching (基于深度学习的地图匹配)。

前五天你已经掌握了:

1. Tensor: 数据载体。
2. Dataset/Loader: 批量搬运数据。
3. Model: 线性层和 LSTM。
4. Inference: 实时滑动窗口。

今天我们要解决智能驾驶定位中最核心的问题: “我有一段 GPS 轨迹, 地图周围有 3 条路, 我到底在哪条路上?”

这不再是一个简单的分类问题（不是单纯判断左转/右转），而是一个**相似度匹配问题 (Similarity Matching)**。

我们要搭建著名的 **孪生网络 (Siamese Network)**。

核心概念：孪生网络 (Siamese Network)

想象你在玩“找茬”游戏，或者比对指纹：

- **左眼看** GPS 轨迹。
- **右眼看** 地图上的道路形状。
- **大脑** 比较两者的特征，算出一个分数：**0 (完全不像)** 到 **1 (一模一样)**。

在代码中，这实现起来非常巧妙：我们要定义一个 LSTM 网络，但是让它**跑两次**（共享权重）。

第一步：构造“成对”数据集 (MatchingDataset)

我们需要的数据格式是 **Pairs (成对数据)**：

- **输入 1:** 车辆实际跑出的 GPS 轨迹（带噪声）。
- **输入 2:** 地图数据库里的道路形状（完美线条）。
- **标签 (Target):**
 - **1** : 如果这两者是同一条路。
 - **0** : 如果这两者不是同一条路。

第二步：搭建孪生网络 (The Siamese Model)

这是今天的重头戏。

第三步：训练 (BCELoss)

因为输出是 0~1 的概率，我们使用 **BCELoss (Binary Cross Entropy)**，这是二分类任务的标配。

```
model.py

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  from torch.utils.data import Dataset, DataLoader
5  import numpy as np
6
7  # --- 1. 定义匹配数据集 ---
8  class MapMatchingDataset(Dataset):
9      def __init__(self, num_samples=2000, seq_len=20):
10         self.num_samples = num_samples
11         self.seq_len = seq_len
```

```

12 self.t = np.linspace(0, 10, seq_len)
13
14 def __len__(self):
15     return self.num_samples
16
17 def __getitem__(self, idx):
18     # 策略: 50% 匹配(1), 50% 不匹配(0)
19     is_match = (np.random.rand() > 0.5)
20
21     # --- 1. 动态生成“底图道路” ---
22     # 我们不再预先存死 roads, 而是现场生成, 这样可以随机覆盖左转和右转
23
24     # 随机决定是“直行”还是“弯道”
25     if np.random.rand() > 0.5:
26         # 直行:  $y = 0$ 
27         map_road = np.stack([self.t, np.zeros_like(self.t)], axis=1)
28         road_type = 'straight'
29     else:
30         # 弯道:  $y = a * x^2$ 
31         # 【关键修改】: 这里的 curvature (曲率) 随机正负!
32         # 随机范围在 -0.1 到 0.1 之间, 不再只是 0.1
33         curvature = np.random.uniform(-0.15, 0.15)
34         # 避免产生太平直的弯道 (如果不幸随机到0)
35         if abs(curvature) < 0.05: curvature = 0.1
36
37         map_road = np.stack([self.t, curvature * self.t**2], axis=1)
38         road_type = 'curve'
39
40     # --- 2. 生成 GPS ---
41     if is_match:
42         # 匹配: GPS 基于同一条路
43         base_road = map_road
44         label = 1.0
45     else:
46         # 不匹配: GPS 基于“相反”类型的路
47         if road_type == 'straight':
48             # 如果底图是直的, GPS 就生成一个弯的 (随机左或右)
49             c = np.random.uniform(-0.15, 0.15)
50             if abs(c) < 0.05: c = 0.1
51             base_road = np.stack([self.t, c * self.t**2], axis=1)
52         else:
53             # 如果底图是弯的, GPS 就生成一个直的
54             base_road = np.stack([self.t, np.zeros_like(self.t)], axis=1)
55             label = 0.0
56
57     # 给 GPS 加噪声
58     gps_noise = np.random.normal(0, 0.3, size=base_road.shape)

```

```

59     gps_traj = base_road + gps_noise
60
61     # 转 Tensor
62     seq1 = torch.tensor(gps_traj, dtype=torch.float32)
63     seq2 = torch.tensor(map_road, dtype=torch.float32)
64     target = torch.tensor([label], dtype=torch.float32)
65
66     return seq1, seq2, target
67
68 # 测试一下
69 ds = MapMatchingDataset()
70 s1, s2, lab = ds[0]
71 print(f"GPS Shape: {s1.shape}, Map Shape: {s2.shape}, Label: {lab}")
72
73 class SiameseLSTMMatcher(nn.Module):
74     def __init__(self):
75         super().__init__()
76
77         # --- 子网络 (Feature Extractor) ---
78         # 用于提取轨迹特征，就像“视网膜”
79         # 我们用 LSTM 把 [20, 2] 的序列变成 [64] 的特征向量
80         self.lstm = nn.LSTM(input_size=2, hidden_size=64, batch_first=True)
81
82         # --- 比较网络 (Comparator) ---
83         # 输入：两个特征向量的差异
84         # 输出：0~1 的相似度概率
85         self.classifier = nn.Sequential(
86             nn.Linear(64, 32),
87             nn.ReLU(),
88             nn.Linear(32, 1),
89             nn.Sigmoid() # 关键！把输出压缩到 0-1 之间作为概率
90         )
91
92     def forward_one(self, x):
93         # 辅助函数：只跑一次 LSTM
94         out, _ = self.lstm(x)
95         return out[:, -1, :] # 取最后一个时间步的特征 [Batch, 64]
96
97     def forward(self, x1, x2):
98         # x1: GPS 轨迹
99         # x2: Map 道路形状
100
101         # 1. 孪生结构核心：同一个 LSTM，调用两次！
102         # 这意味着它们共享同一套权重，用同样的“标准”去审视两个输入
103         feat1 = self.forward_one(x1)
104         feat2 = self.forward_one(x2)
105

```

```

106     # 2. 计算差异
107     # 我们可以计算两个向量的绝对差值  $|v1 - v2|$ 
108     # 如果两个形状很像, 这个差值应该接近 0
109     diff = torch.abs(feet1 - feet2)
110
111     # 3. 这里的 diff 是 [Batch, 64]
112     # 把它喂给分类器, 让它判断“这个差异算不算大?”
113     score = self.classifier(diff)
114
115     return score
116
117 # 实例化
118 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
119 model = SiameseLSTMMatcher().to(device)
120 print(model)
121
122 # 数据加载器
123 loader = DataLoader(MapMatchingDataset(num_samples=2000), batch_size=32,
124                     shuffle=True)
125
126 # 优化器和 Loss
127 optimizer = optim.Adam(model.parameters(), lr=0.001)
128 criterion = nn.BCELoss() # 二分类交叉熵
129
130 print(">>> 开始训练地图匹配模型...")
131
132 for epoch in range(10):
133     total_loss = 0
134     for gps_batch, map_batch, label_batch in loader:
135         # 搬运到 GPU
136         gps_batch = gps_batch.to(device)
137         map_batch = map_batch.to(device)
138         label_batch = label_batch.to(device)
139
140         optimizer.zero_grad()
141
142         # 前向传播: 同时喂入两个序列
143         probs = model(gps_batch, map_batch)
144
145         # 计算 Loss
146         loss = criterion(probs, label_batch)
147         loss.backward()
148         optimizer.step()
149
150         total_loss += loss.item()
151
152     print(f"Epoch {epoch+1}: Avg Loss = {total_loss / len(loader):.4f}")

```



```

152
153 print(">>> 训练完成! ")
154
155 # --- 2. 保存模型 (Saving) ---
156 # 我们不仅要保存模型权重, 还要保存 mean 和 std!
157 # 所以我们创建一个字典来打包所有东西
158 checkpoint = {
159     'model_state_dict': model.state_dict(), # 模型的权重 (w, b)
160 }
161
162 save_path = 'day06_model_v1.pth'
163 torch.save(checkpoint, save_path)
164 print(f">>> 模型全套参数已保存至: {save_path}")

```

run_model.py

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4 import time
5 import random
6 import matplotlib.pyplot as plt
7
8 class SiameseLSTMMatcher(nn.Module):
9     def __init__(self):
10         super().__init__()
11
12         # --- 子网络 (Feature Extractor) ---
13         # 用于提取轨迹特征, 就像“视网膜”
14         # 我们用 LSTM 把 [20, 2] 的序列变成 [64] 的特征向量
15         self.lstm = nn.LSTM(input_size=2, hidden_size=64, batch_first=True)
16
17         # --- 比较网络 (Comparator) ---
18         # 输入: 两个特征向量的差异
19         # 输出: 0~1 的相似度概率
20         self.classifier = nn.Sequential(
21             nn.Linear(64, 32),
22             nn.ReLU(),
23             nn.Linear(32, 1),
24             nn.Sigmoid() # 关键! 把输出压缩到 0-1 之间作为概率
25         )
26
27     def forward_one(self, x):
28         # 辅助函数: 只跑一次 LSTM
29         out, _ = self.lstm(x)
30         return out[:, -1, :] # 取最后一个时间步的特征 [Batch, 64]

```

```

31
32     def forward(self, x1, x2):
33         # x1: GPS 轨迹
34         # x2: Map 道路形状
35
36         # 1. 孪生结构核心：同一个 LSTM，调用两次！
37         # 这意味着它们共享同一套权重，用同样的“标准”去审视两个输入
38         feat1 = self.forward_one(x1)
39         feat2 = self.forward_one(x2)
40
41         # 2. 计算差异
42         # 我们可以计算两个向量的绝对差值  $|v1 - v2|$ 
43         # 如果两个形状很像，这个差值应该接近 0
44         diff = torch.abs(feat1 - feat2)
45
46         # 3. 这里的 diff 是 [Batch, 64]
47         # 把它喂给分类器，让它判断“这个差异算不算大？”
48         score = self.classifier(diff)
49
50         return score
51
52     def generate_data():
53         """生成模拟数据（修正版：X=前向，Y=横向）"""
54
55         # 1. 模拟实时 GPS 轨迹（直行，向东/向 X 轴正方向）
56         # 生成 20 个点，X 从 0 到 10
57         t = np.linspace(0, 10, 20)
58
59         # 【修正点 1】注意这里多加了一层括号 () 组成元组
60         # 第一列是 t (X轴)，第二列是 0 (Y轴)
61         current_gps = np.column_stack((t, np.zeros_like(t)))
62
63         # 添加噪声（针对 X 和 Y 都加一点）
64         current_gps += np.random.normal(0, 0.3, current_gps.shape)
65
66         # 2. 候选道路
67
68         # 候选 A：直行道路 (y = 0)
69         # 【修正点 2】同样加括号
70         road_straight = np.column_stack((t, np.zeros_like(t)))
71
72         # 候选 B：左转道路
73         # （在 X 轴前进时，通常定义 Y 正方向为左，或者根据你的习惯定义）
74         # 这里沿用你的逻辑：用曲线改变 Y 值
75         y_left = 0.1 * t**2 # 假设左转是 Y 变大（你可以改成负数，取决于坐标系定义）
76         road_left = np.column_stack((t, y_left))
77

```

```

78 # 候选 C: 右转道路
79 y_right = -0.1 * t**2 # 假设右转是 Y 变小
80 road_right = np.column_stack((t, y_right))
81
82 candidate_roads = {
83     "Candidate_Straight": road_straight,
84     "Candidate_Left": road_left,
85     "Candidate_Right": road_right
86 }
87
88 return current_gps, candidate_roads
89
90 # 2. 可视化模块 (新增)
91 # =====
92 def plot_scenario(current_gps, candidate_roads):
93     """
94     绘制 GPS 轨迹和候选道路
95     """
96     plt.figure(figsize=(10, 6), dpi=100) # 设置画布大小和清晰度
97
98     # --- 绘制候选道路 ---
99     # 定义一些颜色, 方便区分
100     colors = ['green', 'blue', 'orange']
101     line_styles = ['-', '-', '-'] # 都是实线
102
103     # 遍历字典绘制每一条道路
104     for i, (name, road_data) in enumerate(candidate_roads.items()):
105         # road_data[:, 0] 是所有点的 X 坐标
106         # road_data[:, 1] 是所有点的 Y 坐标
107         plt.plot(road_data[:, 0], road_data[:, 1],
108                 label=name, # 图例名称
109                 color=colors[i % len(colors)], # 循环使用颜色
110                 linestyle=line_styles[i % len(line_styles)],
111                 linewidth=2.5, # 线宽一点, 表示是地图基准
112                 alpha=0.7) # 稍微透明一点
113
114     # --- 绘制实时 GPS 轨迹 ---
115     # 使用醒目的红色, 带圆点标记(o)和虚线(--)连接, 表示它是离散的观测值
116     plt.plot(current_gps[:, 0], current_gps[:, 1],
117             'r--o', # 红色虚线带圆点
118             label='Real-time GPS (Noisy)',
119             linewidth=1.5,
120             markersize=6, # 标记点的大小
121             zorder=10) # zorder=10 保证它画在最上层
122
123     # --- 图表设置 ---
124     plt.title("Map Matching Scenario: Which road is the car on?", fontsize=14)

```

```

125 plt.xlabel("X Position (Forward Direction)", fontsize=12)
126 plt.ylabel("Y Position (Lateral Offset)", fontsize=12)
127
128 plt.legend(fontsize=10) # 显示图例
129 plt.grid(True, linestyle='--', alpha=0.6) # 显示网格
130
131 # 【重要】设置坐标轴比例相等
132 # 这样 1米在横向和纵向上看起来长度是一样的, 真实反映弯道形状
133 plt.axis('equal')
134
135 plt.tight_layout() # 自动调整布局防止重叠
136 print("Plot generated. Check the popup window.")
137 plt.show() # 显示图像
138
139
140
141
142 if __name__ == "__main__":
143
144     # 1.模型加载
145     model = SiameseLSTMMatcher()
146     try:
147         checkpoint = torch.load('day06_model_v1.pth')
148         model.load_state_dict(checkpoint['model_state_dict'])
149         print(">>> 模型权重加载成功! ")
150     except:
151         print(">>> [警告] 未找到模型文件, 使用随机初始化模型演示流程")
152
153     model.eval()
154
155     # 2.数据生成
156     gps_data, roads_map = generate_data()
157     print("GPS Shape:", gps_data.shape)
158     print("First 3 points of GPS:\n", gps_data[:3])
159     print(">>> 数据生成完成")
160
161     # 3.模型预测
162     results = {} # 用来存分数
163
164     # 【修正1】不需要 enumerate, 直接遍历 items() 获取 名字 和 数据
165     for name, road_data in roads_map.items():
166
167         # 【修正2】GPS 数据应该来自外部生成的 gps_data, 而不是循环变量 name
168         # 注意: gps_data 是我们之前 generate_data 返回的那个变量
169         gps_tensor = torch.tensor(gps_data, dtype=torch.float32)
170
171         # 【修正3】road_data 现在纯粹是 numpy 数组了, 可以转换

```

```

172 road_tensor = torch.tensor(road_data, dtype=torch.float32)
173
174 # 2. 增加 Batch 维度 (Batch Size, Sequence Length, Features)
175 # 变成 (1, 20, 2)
176 gps_tensor = gps_tensor.unsqueeze(0)
177 road_tensor = road_tensor.unsqueeze(0)
178
179 with torch.no_grad():
180     score = model(gps_tensor, road_tensor)
181
182     final_score = score.item()
183     results[name] = final_score
184     print(f"候选道路: {name:20} | 模型打分: {final_score:.4f}")
185
186 # --- 找出分数最高的 ---
187 # 使用 Python 的 max 函数找出 value 最大的 key
188 best_road = max(results, key=results.get)
189 print("-" * 30)
190 print(f" 最终匹配结果: 车辆在 [{best_road}] 上! ")
191
192 # 画图
193 plot_scenario(gps_data, roads_map)

```

Day 6 知识点解析

1. 为什么叫“孪生”？代码里的 `feat1 = self.forward_one(x1)` 和 `feat2 = self.forward_one(x2)` 是关键。我们没有定义 `lstm_A` 和 `lstm_B`，而是复用了同一个 `self.lstm`。这意味着模型学会了一种通用的“看路”方法，既能看 GPS 也能看地图线。
2. `torch.abs(feat1 - feat2)` 的作用 这是将两个独立的特征向量交互的最简单方式。
 - 如果 GPS 和 Map 很像，`feat1` 和 `feat2` 会很接近，减法结果接近 0。
 - 全连接层 (`classifier`) 会学到：只要输入接近 0，我就输出 1 (Match)；输入很大，我就输出 0 (Mismatch)。

day07

背景：昨天我们只做了一次匹配（拍一张照，问在哪）。但在真实驾驶中，车在动，GPS 是源源不断进来的。我们需要实时判断车辆状态。

核心概念：滑动窗口 (Sliding Window) 由于我们的模型必须吃 20 个点的序列，我们不能来一个点就测一个点。我们需要维护一个“队列”：

- **T=20秒时：**取 `[0~19]` 的点喂给模型。
- **T=21秒时：**剔除第0个点，加入第20个点，取 `[1~20]` 喂给模型。

- **结果：**我们会得到一条**连续变化的置信度曲线**。

任务目标

1. 生成一条**长距离轨迹**（例如 100 个点）：先直行，中间拐个弯，再直行。
2. 设置两条候选路：
 - **路 A (正确)：**跟着车一起拐弯。
 - **路 B (错误)：**一直直行（在拐弯处会偏离）。
3. 实现**滑动窗口推理**，记录每一时刻的分数。
4. **双图可视化：**上面画地图轨迹，下面画分数随时间变化的波形图。

代码实战

请直接在一个新的 Python 文件中运行以下代码（确保目录下有你昨天训练好的

`day06_model_v1.pth`，如果没有，它会使用随机参数演示流程，但分数就没有意义了）。

代码块

```
1  import torch
2  import torch.nn as nn
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6  # =====
7  # 1. 模型定义 (保持与 Day 6 一致)
8  # =====
9  class SiameseLSTMMatcher(nn.Module):
10     def __init__(self):
11         super().__init__()
12         self.lstm = nn.LSTM(input_size=2, hidden_size=64, batch_first=True)
13         self.classifier = nn.Sequential(
14             nn.Linear(64, 32),
15             nn.ReLU(),
16             nn.Linear(32, 1),
17             nn.Sigmoid()
18         )
19
20     def forward_one(self, x):
21         out, _ = self.lstm(x)
22         return out[:, -1, :]
23
24     def forward(self, x1, x2):
25         feat1 = self.forward_one(x1)
```

```

26     feat2 = self.forward_one(x2)
27     diff = torch.abs(feet1 - feat2)
28     score = self.classifier(diff)
29     return score
30
31     # =====
32     # 2. 长距离数据生成 (Long Trajectory)
33     # =====
34     def generate_long_scenario():
35         """
36         生成一个长达 100 个点的行程
37         场景：0-40直行，40-60向左变道/弯道，60-100直行
38         """
39         total_steps = 100
40         t = np.linspace(0, 50, total_steps) # 时间/纵向位移
41
42         # --- 构造真实的 GPS 轨迹 (车走了个 S 型) ---
43         true_y = np.zeros_like(t)
44         # 前 40 个点是真的，y=0
45         # 中间 20 个点 (idx 40-60) 发生偏移
46         # 使用 sigmoid 函数模拟平滑换道/弯道
47         sigmoid_part = 1 / (1 + np.exp(-(t - 25))) # 简单的 S 型过渡
48         true_y = sigmoid_part * 5 # 最终偏移 5 米
49
50         gps_long = np.column_stack((t, true_y))
51         # 加噪声
52         gps_long += np.random.normal(0, 0.15, gps_long.shape)
53
54         # --- 构造候选道路 ---
55
56         # 候选路 A (Correct): 完美匹配这一条 S 型路径
57         road_correct = np.column_stack((t, true_y)) # 理想状态无噪声
58
59         # 候选路 B (Wrong): 一直是直的 (即在中间段会偏离)
60         # 它只在头尾看起来像，中间完全对不上
61         road_wrong = np.column_stack((t, np.zeros_like(t)))
62
63         return gps_long, road_correct, road_wrong
64
65     # =====
66     # 3. 滑动窗口推理引擎
67     # =====
68     def run_sliding_window_inference(model, gps_long, road_correct, road_wrong,
69                                     window_size=20):
70
71         scores_correct = []
72         scores_wrong = []

```

```

72 time_steps = []
73
74 total_len = len(gps_long)
75
76 print(f">>> 开始滑动窗口推理 (总长度 {total_len}, 窗口 {window_size})...")
77
78 # 从第 20 个点开始, 每次往后滑 1 格
79 for i in range(window_size, total_len):
80     # 1. 切片 (Slicing) - 取过去 20 个点
81     # slice_gps: [20, 2]
82     slice_gps = gps_long[i-window_size : i]
83     slice_road_correct = road_correct[i-window_size : i]
84     slice_road_wrong = road_wrong[i-window_size : i]
85
86     # 2. 转 Tensor 并增加 Batch 维度 -> [1, 20, 2]
87     t_gps = torch.tensor(slice_gps, dtype=torch.float32).unsqueeze(0)
88     t_road_c = torch.tensor(slice_road_correct,
89                             dtype=torch.float32).unsqueeze(0)
90     t_road_w = torch.tensor(slice_road_wrong,
91                             dtype=torch.float32).unsqueeze(0)
92
93     # 3. 推理
94     with torch.no_grad():
95         s_c = model(t_gps, t_road_c).item()
96         s_w = model(t_gps, t_road_w).item()
97
98     scores_correct.append(s_c)
99     scores_wrong.append(s_w)
100     time_steps.append(i)
101
102     return time_steps, scores_correct, scores_wrong
103
104 # =====
105 # 4. 动态分析可视化
106 # =====
107
108 def plot_results(gps, road_c, road_w, time_steps, score_c, score_w):
109     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10))
110
111     # --- 子图 1: 地图视角 ---
112     ax1.set_title("Map View: Trajectory vs Candidates")
113     ax1.plot(road_c[:, 0], road_c[:, 1], 'g-', label="Road A (Correct - S
114             shape)", linewidth=3, alpha=0.5)
115     ax1.plot(road_w[:, 0], road_w[:, 1], 'orange', label="Road B (Wrong -
116             Straight)", linewidth=3, alpha=0.5)
117     ax1.plot(gps[:, 0], gps[:, 1], 'r.', label="GPS Trace", markersize=2)
118     ax1.legend()
119     ax1.grid(True)

```



```

115 ax1.set_xlabel("X (Forward)")
116 ax1.set_ylabel("Y (Lateral)")
117
118 # --- 子图 2: 随着时间变化的置信度 ---
119 ax2.set_title("Real-time Confidence Score (Sliding Window)")
120 ax2.plot(time_steps, score_c, 'g-', label="Score for Road A", linewidth=2)
121 ax2.plot(time_steps, score_w, 'orange', label="Score for Road B",
122          linewidth=2)
123
124 # 画一条 0.5 的阈值线
125 ax2.axhline(y=0.5, color='gray', linestyle='--')
126
127 ax2.set_ylim(-0.1, 1.1)
128 ax2.set_xlabel("Time Step")
129 ax2.set_ylabel("Matching Probability")
130 ax2.legend()
131 ax2.grid(True)
132
133 plt.tight_layout()
134 plt.show()
135
136 # =====
137 # 主程序
138 # =====
139 if __name__ == "__main__":
140     # 1. 加载模型
141     model = SiameseLSTMMatcher()
142     try:
143         # 请确保这个路径是你昨天重新训练过的那个模型
144         model.load_state_dict(torch.load('day06_model_v1.pth')
145                                ['model_state_dict'])
146         print(">>> 模型加载成功!")
147     except:
148         print(">>> [警告] 未找到模型, 使用随机参数演示 (结果会是乱的)")
149
150     model.eval()
151
152     # 2. 生成数据
153     gps, road_correct, road_wrong = generate_long_scenario()
154
155     # 3. 运行滑动窗口
156     steps, sc, sw = run_sliding_window_inference(model, gps, road_correct,
157                                                  road_wrong)
158
159     # 4. 画图分析
160     plot_results(gps, road_correct, road_wrong, steps, sc, sw)

```

运行后请观察图片：

1. 上半张图（地图）：

- 你应该看到红色的 GPS 轨迹像一条蛇一样，先直行，然后随着绿线（S型路）弯曲，最后又直行。
- 橙色线（直路）在中间段被 GPS 甩开了。

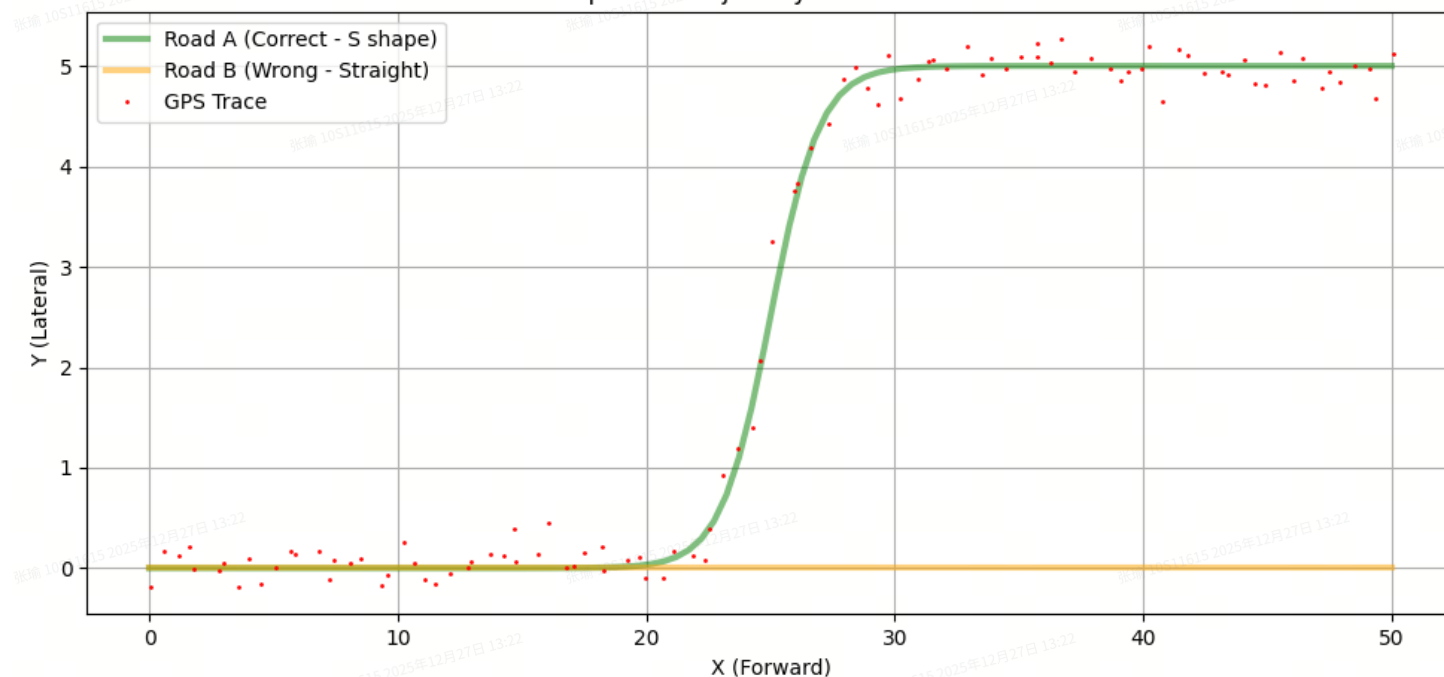
2. 下半张图（分数曲线）：

- **绿色曲线 (Road A)**：应该全程保持在 **0.9 ~ 1.0** 之间（因为它是对的路）。
- **橙色曲线 (Road B)**：
 - 在开始（0-30步）应该是高的（因为那时大家都是直的）。
 - **在中间（30-70步）应该暴跌**（因为路直着走，但车弯出去了，形状对不上了）。
 - 在结尾（70-100步）可能又会回升（因为车又回到了直行状态，虽然Y轴有偏差，但局部形状又是直的了）。

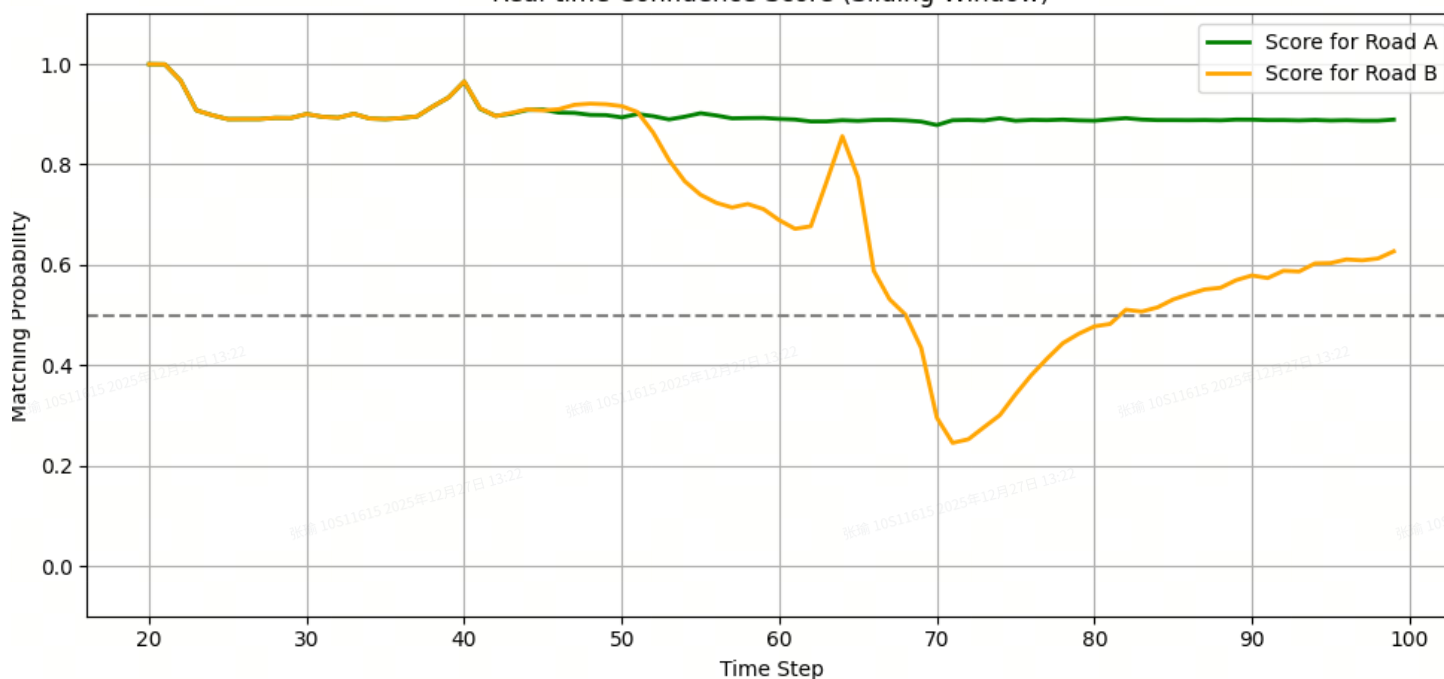
思考题

运行代码后，请看下半张图的橙色曲线。 **问题：** 为什么在最后阶段（70-100步），虽然车已经在橙色道路的上方很远了（Y轴偏移很大），但橙色道路的分数的**可能会**回升？（提示：回忆一下 LSTM 主要是提取什么特征？是绝对坐标，还是形状序列？）

Map View: Trajectory vs Candidates



Real-time Confidence Score (Sliding Window)



1.为什么分数会回升？（核心逻辑）

是的，“只看20帧” (Sliding Window) 是根本原因，这导致模型**“目光短浅”**。

- **中间阶段 (40-60步):** 车在拐弯 (S型)，错误的路是直的。
 - **形状不同:** 弯道 vs 直线。
 - **模型判定:** 形状不匹配! 分数暴跌。 
- **最后阶段 (70-100步):** 车已经拐完了，正在新的车道上**直行**。错误的路虽然在很远的地方，但也是**直行**。
 - **形状相同:** 车走的轨迹是一条直线，路也是一条直线。

- **局部视角**：在一个 20 帧的窗口里，模型看到的只是“一组点排成一条线”。虽然它们的 Y 坐标不同（比如车在 $y=5$ ，路在 $y=0$ ），但它们的**“几何特征”（趋势、曲率）是一模一样的**！
- **模型判定**：既然都是直的，那这就很像啊！分数回升。 

总结：LSTM 这种序列模型非常擅长捕捉**“形状变化”（是不是在拐弯），但往往对“绝对位置偏移”（是不是偏离了 5 米）不够敏感，除非我们在训练数据里刻意强化这一点。这就是为什么在自动驾驶中，我们除了用 LSTM 判别形状，通常还要结合几何距离计算**来做最终决策。

2.20 帧在哪里配置的？（代码定位）

这个配置是在**数据准备阶段**（Dataset）定死的，而不是在 LSTM 模型定义里。

请看你的 `MapMatchingDataset` 类：

关于 LSTM 的 `input_size`：在模型定义里：`self.lstm = nn.LSTM(input_size=2, ...)`
这里的 `2` 是指 `(x, y)` 两个特征。LSTM 并不限制序列长度（Time Steps），你喂给它 20 帧它就吃 20 帧，喂给它 100 帧它就吃 100 帧。**限制来自于我们喂给它的数据切片大小。**

💡 下一步思考

如果我想让模型**极其严格**，即使形状一样，只要 Y 轴偏离了 2 米以上就判为“不匹配”，我该怎么办？（提示：这不需要改模型结构，只需要改训练数据的 Label 策略或输入特征。）

既然不能改模型结构（还得是那个 LSTM），那么我们必须通过**数据策略**来“强迫”模型关注 Y 轴的绝对偏移。

目前的模型之所以认为“平行的两条直路”是匹配的，是因为在它的**“错题本”**（训练集的负样本）里，只有“直路配弯路”是错的。它从来没见过“直路配平行的直路”也是错的，所以它理所当然地认为：“只要形状对上了，就是对的”。

核心解法：难例挖掘 (Hard Negative Sampling)

我们需要在训练数据生成器 (`getitem`) 中，专门加入一种新的**“陷阱”负样本**：

- **输入**：形状完全一样（都是直行，或者都是弯道），但是 Y 轴相差很大（比如 5 米）。
- **Label**：0 (不匹配)。

只要给模型看过这种数据，它立刻就会学到：“哦！原来光长得像还不行，还得靠得近才行！”

代码修改指南

请直接修改昨天的 `MapMatchingDataset` 类中的 `getitem` 函数。

修改后的逻辑：

1. **50% 正样本**：同形状，位置重合（加微小噪声）。
2. **50% 负样本**：
 - 一半是**形状不同**（原来的逻辑：直配弯）。
 - 一半是**形状相同但位置偏移**（新的逻辑：平移 2 米以上）。

day08

Day 8 挑战：处理真实的 GPX 数据与坐标转换

在现实的自动驾驶或地图匹配项目中，你绝对不会拿到 $(x=0, y=5)$ 这种方便的米制坐标。你会收到这样的东西：

- **Latitude (纬度):** 39.9042° N
- **Longitude (经度):** 116.4074° E

你的模型吃不下这些！

1. **单位不统一：**经纬度是角度，不是距离。
2. **比例不同：**在地球上，纬度变化 1 度对应的距离几乎是恒定的（约 111km），但经度变化 1 度对应的距离随纬度变化（在赤道最长，在极点为 0）。
3. **LSTM 需要米：**我们的模型是基于“形状”和“距离”训练的，它需要输入的单位是**米 (Meters)**。

今天的任务：

编写一个完整的预处理管道：

1. **解析：**读取原始的 GPX (XML) 格式数据。
2. **投影 (Projection)：**把球面的经纬度转换成平面的 X/Y 坐标（米）。
3. **可视化：**画出真实的轨迹形状。

核心知识点：简单的“地球展平”术

要进行极其精确的地图测绘，我们需要使用 **UTM 投影**（通用横轴墨卡托）。但在城市范围的地图匹配中，我们通常使用一种**局部切面近似法 (Local Tangent Plane)**。

公式如下（不用死记，理解原理即可）：

设起始点（原点）为 (lat_0, lon_0) ，当前点为 (lat, lon) 。

1. Y 轴（南北向距离）：
- 2.
3. $Y = (lat - lat_0) \times 111,000$
- 4.
5. (因为地球经线上每度大约 111 公里)
6. X 轴（东西向距离）：
- 7.
8. $X = (lon - lon_0) \times 111,000 \times \cos(lat_{radians})$
- 9.

10. (因为纬线圈随着纬度变高而收缩, 需要乘以纬度的余弦值)

实战代码

由于我无法给你发送文件, 我在代码里硬编码了一段真实的北京二环附近的 GPX 数据字符串。

请运行以下代码:

代码块

```
1  import xml.etree.ElementTree as ET
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import math
5
6  # =====
7  # 1. 模拟真实的 GPX 文件内容 (XML格式)
8  # =====
9  gpx_data_content = """
10 <gpx version="1.1">
11   <trk>
12     <name>Real World Test Drive</name>
13     <trkseg>
14       <trkpt lat="39.98470" lon="116.31841"></trkpt>
15       <trkpt lat="39.98482" lon="116.31838"></trkpt>
16       <trkpt lat="39.98501" lon="116.31835"></trkpt>
17       <trkpt lat="39.98555" lon="116.31810"></trkpt>
18       <trkpt lat="39.98610" lon="116.31790"></trkpt>
19       <trkpt lat="39.98688" lon="116.31755"></trkpt>
20       <trkpt lat="39.98750" lon="116.31720"></trkpt>
21       <trkpt lat="39.98820" lon="116.31690"></trkpt>
22       <trkpt lat="39.98850" lon="116.31750"></trkpt>
23       <trkpt lat="39.98860" lon="116.31880"></trkpt>
24       <trkpt lat="39.98865" lon="116.31990"></trkpt>
25       <trkpt lat="39.98870" lon="116.32100"></trkpt>
26       <trkpt lat="39.98872" lon="116.32250"></trkpt>
27       <trkpt lat="39.98875" lon="116.32350"></trkpt>
28     </trkseg>
29   </trk>
30 </gpx>
31 """
32
33 # =====
34 # 2. 解析器 (Parsing)
35 # =====
36 def parse_gpx(xml_string):
37     root = ET.fromstring(xml_string)
```

```

38 points = []
39
40 # 查找所有的 trkpt (Track Point) 标签
41 # namespace 处理有时候很麻烦，这里假设是最简结构
42 for trkpt in root.findall('./trkpt'):
43     lat = float(trkpt.get('lat'))
44     lon = float(trkpt.get('lon'))
45     points.append((lat, lon))
46
47     return np.array(points)
48
49 # =====
50 # 3. 坐标转换器 (Lat/Lon -> Meters)
51 # =====
52 def latlon_to_meters(gps_points):
53     """
54     将经纬度列表转换为以第一个点为原点的局部平面坐标 (ENU)
55     """
56     origin_lat = gps_points[0][0]
57     origin_lon = gps_points[0][1]
58
59     # 将原点纬度转换为弧度，用于计算 x 轴的压缩比
60     lat_rad = math.radians(origin_lat)
61
62     # 地球半径常量 (简化版)
63     DEG_TO_M_LAT = 111000 # 纬度 1度 ≈ 111km
64     DEG_TO_M_LON = 111000 * math.cos(lat_rad) # 经度 1度 ≈ 111km * cos(lat)
65
66     xy_points = []
67
68     for lat, lon in gps_points:
69         # 1. 减去原点 (Delta)
70         d_lat = lat - origin_lat
71         d_lon = lon - origin_lon
72
73         # 2. 转换为米
74         pos_y = d_lat * DEG_TO_M_LAT
75         pos_x = d_lon * DEG_TO_M_LON
76
77         xy_points.append([pos_x, pos_y])
78
79     return np.array(xy_points)
80
81 # =====
82 # 主程序
83 # =====
84 if __name__ == "__main__":

```

```

85 # 1. 解析
86 print(">>> 解析 GPX 数据...")
87 raw_gps = parse_gpx(gpx_data_content)
88 print(f"原始数据 (Lat/Lon) 前3个点:\n{raw_gps[:3]}")
89
90 # 2. 投影转换
91 print("\n>>> 执行坐标投影 (Lat/Lon -> Meters)...")
92 xy_gps = latlon_to_meters(raw_gps)
93 print(f"投影数据 (Meters) 前3个点:\n{xy_gps[:3]}")
94
95 # 3. 可视化
96 plt.figure(figsize=(10, 5))
97
98 # 左图: 原始经纬度 (看起来虽然也像, 但在数学上是变形的)
99 plt.subplot(1, 2, 1)
100 plt.plot(raw_gps[:, 1], raw_gps[:, 0], 'r-o')
101 plt.title("Raw Lat/Lon (Degrees)")
102 plt.xlabel("Longitude")
103 plt.ylabel("Latitude")
104 plt.grid(True)
105 # 强制不自动调整比例, 看看会发生什么 (通常看起来会很扁或很长)
106
107 # 右图: 米制坐标 (这是真实的物理形状)
108 plt.subplot(1, 2, 2)
109 plt.plot(xy_gps[:, 0], xy_gps[:, 1], 'b-o')
110 plt.title("Projected Local Plane (Meters)")
111 plt.xlabel("X (East-West) meters")
112 plt.ylabel("Y (North-South) meters")
113 plt.grid(True)
114 plt.axis('equal') # 关键! 保持 1米:1米 的比例
115
116 plt.tight_layout()
117 plt.show()

```

这说明你的观察非常仔细! 没看出来变化其实是**正常的**, 原因在于我们这次用的数据恰好长宽比较接近 (大约 400米 × 500米), 就像一个正方形, 所以默认拉伸后变形不明显。

为了让你直观地看到如果不加 `equal` 会有多危险, 我们需要一个**长宽比极度失衡**的例子。

视觉实验: 把路拉长!

想象一下, 如果我们的车在一个**很窄的直道上只变了一次道**。

- **X轴 (横向) 变化:** 只有 3 米 (变道)。
- **Y轴 (纵向) 变化:** 开了 1000 米。

如果没有 `axis('equal')`，Matplotlib 会强行把这 3 米拉得跟 1000 米一样长，填满整个正方形画布。

请运行这段代码，你就明白“变形”有多可怕了：

代码块

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def plot_distortion_test():
5     # 模拟数据：在直路上变道
6     # Y 轴：走了 1000 米
7     y = np.linspace(0, 1000, 100)
8     # X 轴：慢慢变道，横向只移动了 5 米
9     x = 5 * (1 / (1 + np.exp(-(y - 500)/50)))
10
11     plt.figure(figsize=(10, 4))
12
13     # 1. 错误的画法 (Auto Scaling - 默认)
14     plt.subplot(1, 2, 1)
15     plt.plot(x, y, 'r-', linewidth=3)
16     plt.title("No 'equal' (Distorted)")
17     plt.grid(True)
18     # Matplotlib 会把 X轴的 0-5米 拉伸得和 Y轴的 0-1000米 一样长！
19     # 看起来像是在过一个 45度 的超级大急弯！
20
21     # 2. 正确的画法 (Equal Aspect Ratio)
22     plt.subplot(1, 2, 2)
23     plt.plot(x, y, 'g-', linewidth=3)
24     plt.title("With 'equal' (True Shape)")
25     plt.grid(True)
26     plt.axis('equal')
27     # 强制 1米 X轴长度 = 1米 Y轴长度
28     # 看起来就是一条几乎直的线（这是真相）
29
30     plt.tight_layout()
31     plt.show()
32
33 plot_distortion_test()
```

这说明你的观察非常仔细！没看出来变化其实是**正常的**，原因在于我们这次用的数据恰好长宽比较接近（大约 400米 × 500米），就像一个正方形，所以默认拉伸后变形不明显。

为了让你直观地看到如果不加 `equal` 会有多危险，我们需要一个**长宽比极度失衡**的例子。

视觉实验：把路拉长！

想象一下，如果我们的车在一个很窄的直道上只变了一次道。

- **X轴（横向）变化：**只有 3 米（变道）。
- **Y轴（纵向）变化：**开了 1000 米。

如果没有 `axis('equal')`，Matplotlib 会强行把这 3 米拉得跟 1000 米一样长，填满整个正方形画布。

请运行这段代码，你就明白“变形”有多可怕了：

Python

代码块

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def plot_distortion_test():# 模拟数据：在直路上变道# Y 轴：走了 1000 米
5     y = np.linspace(0, 1000, 100)
6     # X 轴：慢慢变道，横向只移动了 5 米
7     x = 5 * (1 / (1 + np.exp(-(y - 500)/50)))
8
9     plt.figure(figsize=(10, 4))
10
11     # 1. 错误的画法 (Auto Scaling - 默认)
12     plt.subplot(1, 2, 1)
13     plt.plot(x, y, 'r-', linewidth=3)
14     plt.title("No 'equal' (Distorted)")
15     plt.grid(True)
16     # Matplotlib 会把 X轴的 0-5米 拉伸得和 Y轴的 0-1000米 一样长！# 看起来像是在过一个 45度 的超级大急弯！
17     # 2. 正确的画法 (Equal Aspect Ratio)
18     plt.subplot(1, 2, 2)
19     plt.plot(x, y, 'g-', linewidth=3)
20     plt.title("With 'equal' (True Shape)")
21     plt.grid(True)
22     plt.axis('equal')
23     # 强制 1米 X轴长度 = 1米 Y轴长度# 看起来就是一条几乎直的线（这是真相）
24
25     plt.tight_layout()
26     plt.show()
27 plot_distortion_test()
```

这对 LSTM 意味着什么风险？

你在图中看到的**“视觉欺骗”，如果发生数据预处理阶段，就是对模型的“数学欺骗”**。

这对应着深度学习中一个极易犯的错误：**独立归一化 (Independent Normalization)**。

❌ 错误的预处理（模型会瞎）

如果你对 X 轴和 Y 轴分别做标准的归一化 ($(x - \mu) / \sigma$)：

- **X轴 (横向)**：虽然只动了 5 米，但归一化后变成了 -1 到 1。
- **Y轴 (纵向)**：虽然动了 1000 米，归一化后也变成了 -1 到 1。

后果：

在 LSTM 眼里，X 的变化幅度和 Y 一样大！

- **真实情况**：车子稍微动了一下方向盘（微调）。
- **模型看到**：车子在漂移过大弯！🚗💨

✅ 正确的预处理（保持比例）

对于地图数据，我们必须使用全局缩放。

即：找出整个地图中最大的那个范围（Max Range），然后 X 和 Y 都除以同一个数字。这样 5 米依然是很小的数，1000 米依然是很大的数，**几何形状（Geometry）**才被保留了下来。

总结：`plt.axis('equal')` 不仅仅是为了画图好看，它提醒我们在处理空间数据时，**必须严守 X 和 Y 的物理比例一致性**，否则模型学到的形状就是扭曲的。

day09

Day 9 挑战：海量候选检索与 R-Tree 思想

场景：假设你在北京二环上开车。

- **你的 GPS**：位于 `(X=5000, Y=5000)`。
- **地图数据库**：包含了全中国 **2000 万条** 道路数据。

问题：如果我们用 `for` 循环把这 2000 万条路都喂给 LSTM 模型算一遍分数……

- CPU 会爆炸。💣
- 车都开到五环了，二环的结果还没算出来。

解决方案：漏斗筛选 (The Funnel) 我们不能直接让 LSTM 处理全量数据。我们需要一个极快的前置步骤：

1. **第一层（空间索引）**：快速找出 GPS 周围 **100 米** 范围内的路。（可能剩下 5 条）
2. **第二层（精细匹配）**：只把这 5 条路喂给 LSTM。（耗时毫秒级）

今天我们不依赖复杂的数据库（如 PostgreSQL/PostGIS），而是用 Python 手写一个**基于 Bounding Box (包围盒)**的核心检索逻辑。

核心概念：AABB 包围盒

为了判断“两条线是否靠近”，计算几何距离（欧氏距离）其实是很慢的（因为要算平方根）。

计算机图形学和地图引擎中最常用的技巧是 **AABB (Axis-Aligned Bounding Box)**:

- 不管道路形状多复杂, 我先给它套一个矩形框 (最小 x, 最大 x, 最小 y, 最大 y)。
- **判断逻辑**: 如果两个矩形框都没有重叠, 那里面的道路绝对不可能相交或靠近。
- **速度**: 只需要比较 `x < x_max` 这种大小关系, 比算距离快 100 倍。

实战代码: 编写空间检索引擎

我们将生成 50 条随机道路 (模拟城市路网), 然后给出一个 GPS 轨迹, 看算法能不能瞬间把周围的“候选路”圈出来。

请运行以下代码:

代码块

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.patches as patches
4
5 # =====
6 # 1. 模拟地图数据库 (Map Database)
7 # =====
8 class RoadSegment:
9     def __init__(self, id, points):
10         self.id = id
11         self.points = points # Shape: [N, 2]
12
13         # --- 核心: 预计算包围盒 (Min/Max X, Min/Max Y) ---
14         # 这一步通常在地图加载时做一次, 之后查询就很快
15         self.min_x = np.min(points[:, 0])
16         self.max_x = np.max(points[:, 0])
17         self.min_y = np.min(points[:, 1])
18         self.max_y = np.max(points[:, 1])
19
20 def generate_city_map(num_roads=50):
21     roads = []
22     for i in range(num_roads):
23         # 随机生成一些散落在 (0,0) 到 (1000,1000) 区域的路
24         start_x = np.random.uniform(0, 1000)
25         start_y = np.random.uniform(0, 1000)
26
27         # 让路稍微长一点, 随机延伸
28         length = np.random.uniform(50, 200)
29         angle = np.random.uniform(0, 2 * np.pi)
30
31         end_x = start_x + length * np.cos(angle)
```

```

32     end_y = start_y + length * np.sin(angle)
33
34     # 插值生成几个点, 模拟一条路
35     t = np.linspace(0, 1, 10)
36     x = start_x + (end_x - start_x) * t
37     y = start_y + (end_y - start_y) * t
38
39     road_points = np.column_stack((x, y))
40     roads.append(RoadSegment(f"Road_{i}", road_points))
41     return roads
42
43     # =====
44     # 2. 空间检索引擎 (Spatial Indexer)
45     # =====
46     def find_candidates(gps_trace, all_roads, buffer_radius=50):
47         """
48         输入:
49             gps_trace: 当前的 20个 GPS 点
50             all_roads: 整个城市的道路列表
51             buffer_radius: 搜索半径 (比如只看周围 50米)
52         输出:
53             candidates: 落入搜索范围的道路列表
54             search_box: 搜索框坐标 (用于画图)
55         """
56         candidates = []
57
58         # 1. 计算 GPS 轨迹的包围盒
59         g_min_x = np.min(gps_trace[:, 0])
60         g_max_x = np.max(gps_trace[:, 0])
61         g_min_y = np.min(gps_trace[:, 1])
62         g_max_y = np.max(gps_trace[:, 1])
63
64         # 2. 向外扩充半径 (Buffer) -> 形成搜索框 (Search Box)
65         search_min_x = g_min_x - buffer_radius
66         search_max_x = g_max_x + buffer_radius
67         search_min_y = g_min_y - buffer_radius
68         search_max_y = g_max_y + buffer_radius
69
70         search_box = (search_min_x, search_max_x, search_min_y, search_max_y)
71
72         # 3. 快速筛选 (Broad Phase)
73         # 遍历所有路, 看它们的盒子是否和搜索框相交
74         for road in all_roads:
75             # AABB 重叠测试算法:
76             # 如果 A的左边 > B的右边, 或者 A的右边 < B的左边... 也就是完全没挨着
77             if (road.min_x > search_max_x or road.max_x < search_min_x or
78                 road.min_y > search_max_y or road.max_y < search_min_y):

```

```

79         continue # 绝对不相交, 跳过
80
81     # 如果盒子相交, 暂时认为是候选 (虽然可能只是擦肩而过, 但足以进入下一轮 LSTM)
82     candidates.append(road)
83
84     return candidates, search_box
85
86 # =====
87 # 3. 主程序与可视化
88 # =====
89 if __name__ == "__main__":
90     # 1. 造城
91     city_roads = generate_city_map(num_roads=50)
92
93     # 2. 造一个 GPS 轨迹 (假装我们在地图中间)
94     gps_center = np.array([[500, 500]])
95     gps_trace = gps_center + np.random.normal(0, 10, (20, 2)) # 20个点的一团
96
97     # 3. 执行检索 (这是今天的重点!)
98     found_roads, s_box = find_candidates(gps_trace, city_roads,
99                                         buffer_radius=60)
100
101     print(f"地图总道路数: {len(city_roads)}")
102     print(f"筛选出的候选道路数: {len(found_roads)}")
103     print("候选ID:", [r.id for r in found_roads])
104
105     # 4. 可视化
106     fig, ax = plt.subplots(figsize=(8, 8))
107
108     # 画所有路 (灰色, 表示未选中)
109     for road in city_roads:
110         ax.plot(road.points[:, 0], road.points[:, 1], c='lightgray', lw=1,
111                 zorder=1)
112
113     # 画候选路 (蓝色, 加粗)
114     for road in found_roads:
115         ax.plot(road.points[:, 0], road.points[:, 1], c='blue', lw=2,
116                 zorder=2, label='Candidates')
117
118     # 画 GPS (红色)
119     ax.plot(gps_trace[:, 0], gps_trace[:, 1], 'r.', zorder=3, label='GPS')
120
121     # 画搜索框 (红色虚线矩形)
122     # s_box = (min_x, max_x, min_y, max_y)
123     rect_width = s_box[1] - s_box[0]
124     rect_height = s_box[3] - s_box[2]
125     rect = patches.Rectangle((s_box[0], s_box[2]), rect_width, rect_height,

```

```

123             linewidth=2, edgecolor='red', facecolor='none',
            linestyle='--', label='Search Buffer')
124     ax.add_patch(rect)
125
126     # 避免图例重复
127     handles, labels = plt.gca().get_legend_handles_labels()
128     by_label = dict(zip(labels, handles))
129     ax.legend(by_label.values(), by_label.keys())
130
131     ax.set_title("Spatial Indexing: Finding Candidates Efficiently")
132     ax.set_xlim(0, 1000)
133     ax.set_ylim(0, 1000)
134     plt.show()

```

🧐 代码解析

1. RoadSegment 类：

- 我们在初始化时就计算好了 `min_x, max_x...`。这叫**预计算 (Pre-computation)**。在实际工程中，这些数值会存储在数据库的索引列中。

2. find_candidates 函数：

- 它没有计算 `sqrt((x1-x2)^2 + ...)` 这种昂贵的距离公式。
- 它只用了 4 次比较运算 (`> < > <`)。这在 CPU 层面是极快的。
- 逻辑是：“宁可错杀一千，不可放过一个”。只要盒子碰到了，就先算作候选。把精细筛选的工作留给后面的 LSTM 或者更精细的几何算法。

3. 可视化：

- 你会看到一个**红色的虚线框**。
- 所有穿过这个框、或者在框里面的线，都变蓝了。
- 外面的线全是灰色的。

🔗 将 Day 6-9 串联起来

现在你脑海中应该有了完整的 **Map Matching Pipeline**：

- 输入**：接收原始 GPX 经纬度数据。
- 投影 (Day 8)**：转为米制平面坐标 (x, y) 。
- 空间检索 (Day 9)**：画一个框，从 2000 万条路中筛出 5 条 **Candidate Roads**。
- 模型推理 (Day 6/7)**：
 - 把 GPS 和这 5 条路分别配对。
 - 喂给 **Siamese LSTM**。

- 得到 5 个分数。

5. 决策：取 `argmax`，分数最高的那个就是车所在的道路。

思考题

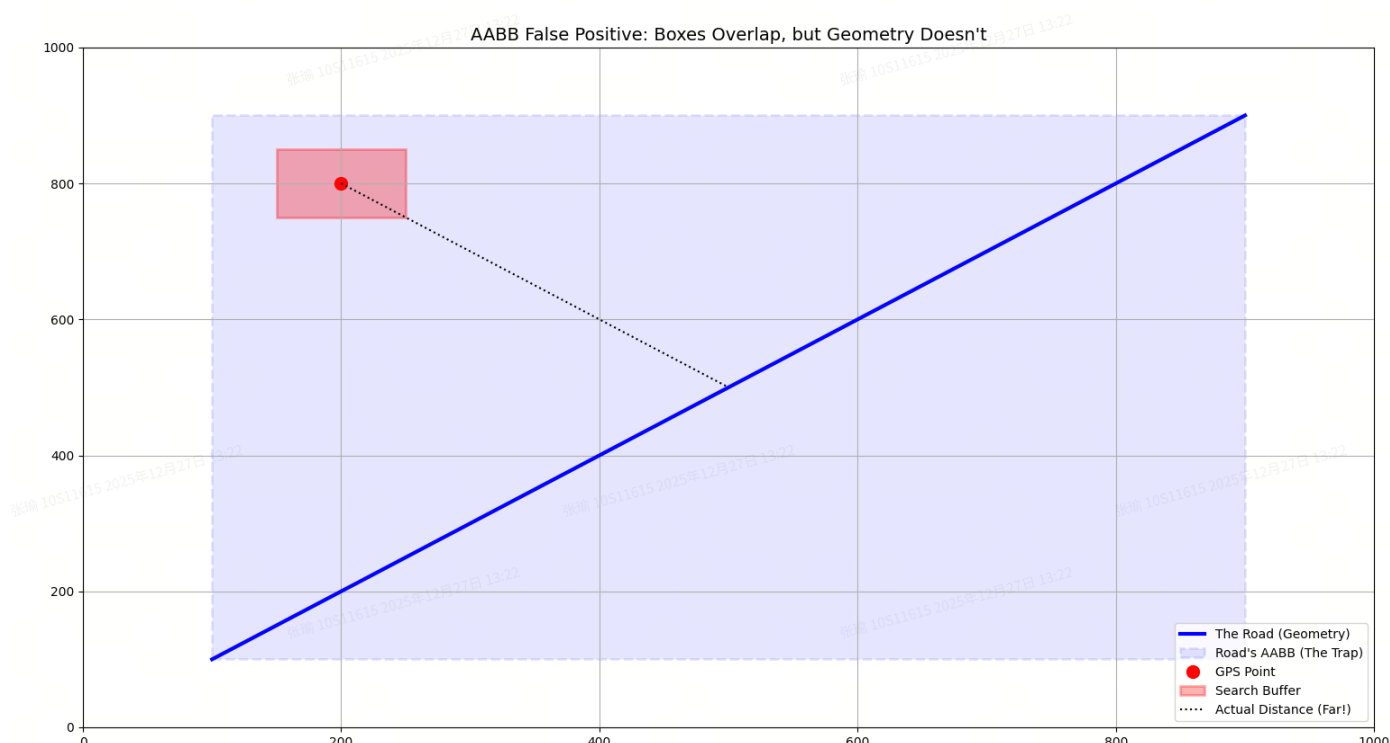
运行代码多试几次。

你会发现一种特殊情况：有一条路，它本身没有穿过红框，但是它的对角线包围盒穿过了红框，导致它也被选中了（变蓝了），虽然它其实离得有点远。

这就是 AABB 算法唯一的缺点：有些虚报 (False Positives)。

- 问题：这对我们的系统有致命影响吗？
- 提示：回想一下，选中之后我们会做什么？

(如果你想通了这个问题，你就明白为什么工业界都爱用 AABB 了。)



答案：完全没有致命影响，反而是一种非常健康的策略。

为什么？请记住这个架构：

漏斗模型 (The Funnel)

1. 第一层 (AABB 粗筛)：

- 目标：速度至上！只负责把绝对不可能的路剔除掉。
- 代价：允许混进来几个“滥竽充数”的（比如上面那条斜路）。
- 结果：从 2000 万条 -> 筛选出 5 条（包含 1 条真路，3 条附近的干扰路，1 条上面这种离得远但盒子碰到的“冤大头”）。

2. 第二层 (LSTM 精细模型)：

- **目标：**准确度至上！
- **操作：**模型会计算 GPS 轨迹和这条“冤大头”斜路的相似度。
- **结果：**
 - 模型看到：GPS 在 (200, 800)，路在 (500, 500)。
 - 模型判断：距离太远，形状不对。
 - **打分：** Score = 0.001。

结论：这一条“虚报”的道路，仅仅是**浪费了 LSTM 一次推理的计算资源（几毫秒）**而已。它最终会被 LSTM 给出的低分无情淘汰。

day10

今天，我们要完成最后的拼图：**端到端地图匹配引擎 (The End-to-End Engine)**。

我们面临的最后一个挑战是：“抖动” (Flickering)。

Day 10 挑战：解决“左右横跳”与时间一致性

场景：

前面有一个 Y 字形路口（一条直行，一条微左转）。

- 两条路在分岔口非常接近。
- GPS 有噪声。
- **T=10s**：模型觉得像左转道 (Score 0.51)。
- **T=11s**：GPS 稍微飘了一点，模型觉得像直行道 (Score 0.51)。
- **后果：**你的导航图标会在地图上疯狂地“左右横跳”，用户体验极差。

解决方案：惯性平滑 (Momentum Smoothing)

如果车子上一秒在“左转道”，那么它下一秒大概率还在“左转道”，除非有极强的证据证明它变了。

我们需要引入**“历史记忆”**。

公式：

$$FinalScore_t = \alpha \cdot ModelScore_t + (1 - \alpha) \cdot FinalScore_{t-1}$$

- α 是“信任新数据”的系数（比如 0.3）。
- 这意味着我们保留了 70% 的旧分数，只让新分数影响 30%。这能让结果如丝般顺滑。

🏠 终极代码：完整集成版引擎

这段代码集成了：数据生成 -> 滑动窗口 -> LSTM 模型 -> 惯性平滑。

这也是一个微型的工业级架构原型。

(注：为了方便演示，我们继续使用之前定义的 `SiameseLSTMMatcher` 类结构，假设你已经有了昨天的环境)

代码块

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib.gridspec as gridspec
4
5  # =====
6  # 1. 场景生成器 (Y-Split Scenario)
7  # =====
8  def generate_y_split_scenario():
9      """
10     生成一个 Y 字形路口场景
11     0-40米：重合直行
12     40-100米：分岔
13     """
14     # 生成 100 个时间步
15     t = np.linspace(0, 100, 100)
16
17     # --- 1. 构造道路几何 ---
18     # Road A (Left): 在 x>40 后向左弯 (y 增加)
19     y_left = np.zeros_like(t)
20     mask = t > 40
21     y_left[mask] = 0.05 * (t[mask] - 40)**1.2 # 指数增长，模拟弯道
22     road_left = np.column_stack((t, y_left))
23
24     # Road B (Right): 在 x>40 后向右弯 (y 减少)
25     y_right = np.zeros_like(t)
26     y_right[mask] = -0.05 * (t[mask] - 40)**1.2
27     road_right = np.column_stack((t, y_right))
28
29     # --- 2. 构造 GPS 轨迹 ---
30     # 假设车主要走的是 Left Road，但在分岔口有些犹豫和抖动
31     gps_trace = road_left.copy()
32
33     # 添加较大的随机噪声 (模拟定位漂移)
34     np.random.seed(42) # 固定随机种子，保证每次运行效果一致
35     noise_level = 1.5
36     gps_trace[:, 1] += np.random.normal(0, noise_level, t.shape)
37
38     candidates = {
39         "Road A (Left)": road_left,
40         "Road B (Right)": road_right
41     }
```

```

42
43     return gps_trace, candidates
44
45     # =====
46     # 2. 模拟模型与平滑引擎
47     # =====
48     class SmoothMatchingEngine:
49         def __init__(self, alpha=0.2):
50             self.alpha = alpha
51             self.score_memory = {} # 记忆库
52
53         def mock_model_predict(self, gps_point, road_point):
54             """
55             这里模拟一个 '神经质' 的 AI 模型。
56             它基于距离打分，但是加了很多随机抖动，模拟模型的不稳定性。
57             """
58             # 计算欧氏距离
59             dist = np.linalg.norm(gps_point - road_point)
60
61             # 基础分：距离越近分越高 (Exp decay)
62             base_score = np.exp(-0.2 * dist)
63
64             # 添加 '模型不确定性' 噪声 (Model Uncertainty)
65             # 这模拟了 LSTM 在模糊地带的跳变
66             model_jitter = np.random.normal(0, 0.1)
67             final_score = base_score + model_jitter
68
69             # 限制在 0~1 之间
70             return np.clip(final_score, 0.0, 1.0)
71
72         def step(self, current_gps, current_roads):
73             """
74             处理一帧数据：先推理，再平滑
75             """
76             raw_scores = {}
77             smoothed_scores = {}
78
79             for name, road_pt in current_roads.items():
80                 # 1. 模拟模型推理 (Raw)
81                 raw_s = self.mock_model_predict(current_gps, road_pt)
82                 raw_scores[name] = raw_s
83
84                 # 2. 获取上一时刻的记忆 (Memory)
85                 last_s = self.score_memory.get(name, 0.5) # 默认从 0.5 开始
86
87                 # 3. 惯性平滑公式 (The Momentum)
88                 # alpha 越小，惯性越大，越平滑

```

```

89     smooth_s = self.alpha * raw_s + (1 - self.alpha) * last_s
90
91     # 4. 更新记忆
92     self.score_memory[name] = smooth_s
93     smoothed_scores[name] = smooth_s
94
95     return raw_scores, smoothed_scores
96
97     # =====
98     # 3. 主程序与可视化 (核心优化部分)
99     # =====
100 def main():
101     # 1. 准备数据
102     gps_full, roads = generate_y_split_scenario()
103     engine = SmoothMatchingEngine(alpha=0.15) # 设置较强的平滑系数
104
105     history_raw = {name: [] for name in roads}
106     history_smooth = {name: [] for name in roads}
107     time_steps = range(len(gps_full))
108
109     # 2. 模拟实时运行
110     print(">>> 引擎启动, 处理中...")
111     for t in time_steps:
112         # 取当前时刻的点
113         curr_gps = gps_full[t]
114         # 取当前时刻道路上的对应点 (简化处理, 假设已匹配到最近点)
115         curr_roads = {name: rd[t] for name, rd in roads.items()}
116
117         raw, smooth = engine.step(curr_gps, curr_roads)
118
119         for name in roads:
120             history_raw[name].append(raw[name])
121             history_smooth[name].append(smooth[name])
122
123     # 3. 高级可视化布局 (3个子图)
124     fig = plt.figure(figsize=(14, 10))
125     gs = gridspec.GridSpec(2, 2, height_ratios=[1, 1])
126
127     # --- 图 1: 空间地图 (上部分, 占据整行) ---
128     ax_map = fig.add_subplot(gs[0, :])
129     ax_map.set_title("1. Spatial View: GPS Trace vs. Roads (The Reality)",
130                     fontsize=14, fontweight='bold')
131
132     # 画路
133     colors = {'Road A (Left)': 'green', 'Road B (Right)': 'orange'}
134     for name, rd in roads.items():

```

```

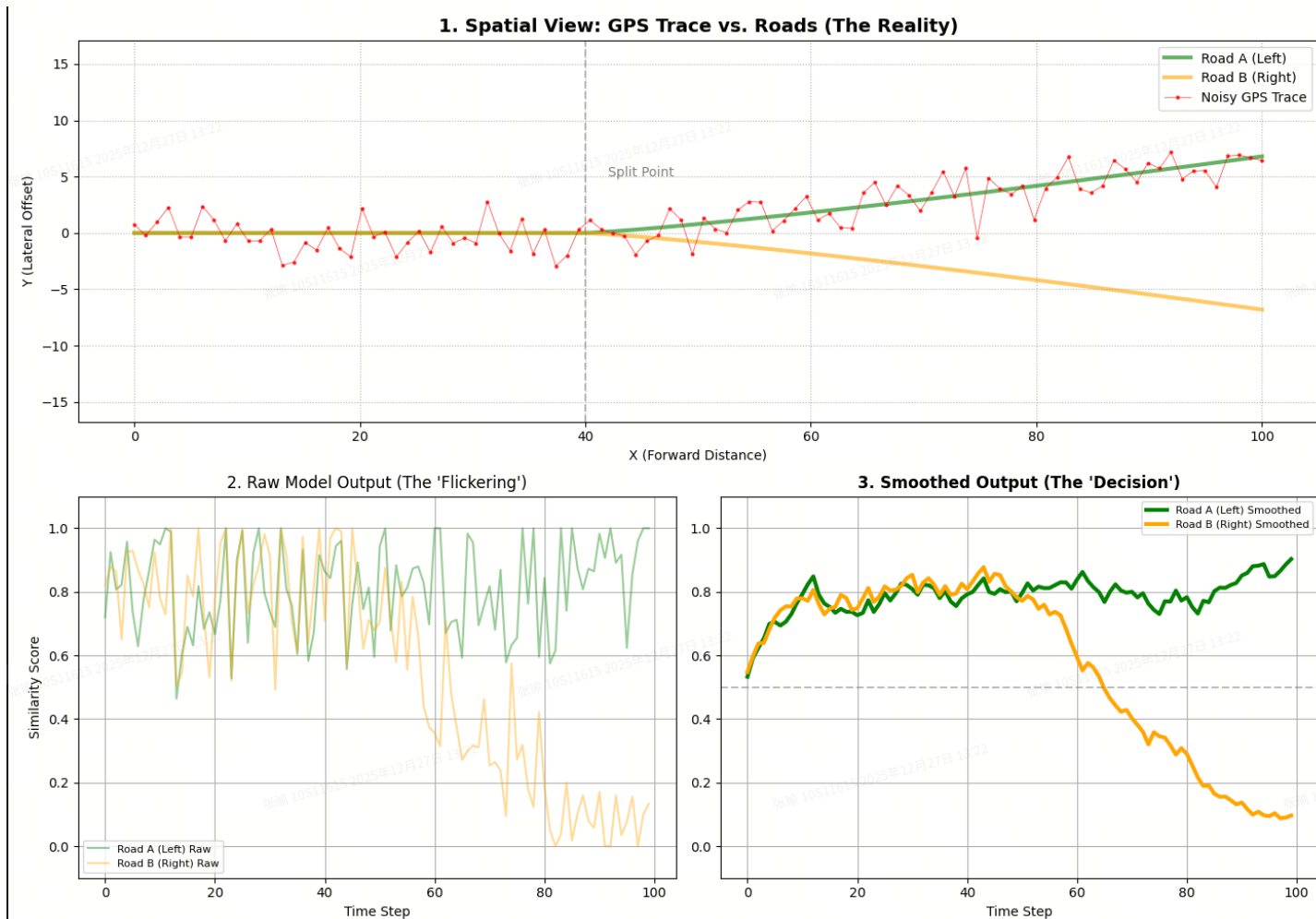
134     ax_map.plot(rd[:, 0], rd[:, 1], color=colors[name], linewidth=3,
135                alpha=0.6, label=name)
136
137     # 画 GPS
138     ax_map.plot(gps_full[:, 0], gps_full[:, 1], 'r.-', linewidth=0.5,
139                markersize=4, alpha=0.8, label="Noisy GPS Trace")
140
141     # 标注分岔区域
142     ax_map.axvline(x=40, color='gray', linestyle='--', alpha=0.5)
143     ax_map.text(42, 5, "Split Point", color='gray')
144
145     ax_map.legend()
146     ax_map.grid(True, linestyle=':')
147     ax_map.set_xlabel("X (Forward Distance)")
148     ax_map.set_ylabel("Y (Lateral Offset)")
149     ax_map.axis('equal') # 关键! 保持比例
150
151     # --- 图 2: 原始分数 (左下) ---
152     ax_raw = fig.add_subplot(gs[1, 0])
153     ax_raw.set_title("2. Raw Model Output (The 'Flickering')", fontsize=12)
154     for name in roads:
155         ax_raw.plot(time_steps, history_raw[name], color=colors[name],
156                    alpha=0.4, linewidth=1.5, label=f"{name} Raw")
157
158     ax_raw.set_ylim(-0.1, 1.1)
159     ax_raw.set_ylabel("Similarity Score")
160     ax_raw.set_xlabel("Time Step")
161     ax_raw.grid(True)
162     ax_raw.legend(fontsize=8)
163
164     # --- 图 3: 平滑分数 (右下) ---
165     ax_smooth = fig.add_subplot(gs[1, 1])
166     ax_smooth.set_title("3. Smoothed Output (The 'Decision')", fontsize=12,
167                        fontweight='bold')
168     for name in roads:
169         # 画粗线, 表示这是最终决策
170         ax_smooth.plot(time_steps, history_smooth[name], color=colors[name],
171                        linewidth=3, label=f"{name} Smoothed")
172
173     # 画 0.5 决策线
174     ax_smooth.axhline(y=0.5, color='gray', linestyle='--', alpha=0.5)
175
176     ax_smooth.set_ylim(-0.1, 1.1)
177     ax_smooth.set_xlabel("Time Step")
178     ax_smooth.grid(True)
179     ax_smooth.legend(fontsize=8)

```

```

176 plt.tight_layout()
177 print(">>> 可视化生成完毕! ")
178 plt.show()
179
180 if __name__ == "__main__":
181     main()

```



深度解析这三张图

运行代码后，你会看到一个非常直观的面板：

顶部的地图 (Spatial View)

- **现象：**你可以看到红色的 GPS 轨迹虽然大致是往左走（跟着绿线），但是**非常抖动**。
- **关键区域 ($X > 40$)：**在分岔后，红色轨迹经常会因为噪声，“跳”到橙色线（右转道）的领地里去。
- **物理意义：**这就是“左右横跳”的物理根源——噪声导致 GPS 坐标在两条路中间摇摆不定。

左下的原始分数 (Raw Output)

- **现象：**这就是我们所说的 **Flickering (闪烁/抖动)**。

- **致命点**：请仔细看 $T=40$ 到 $T=70$ 这个区间。虽然车主要是往左走，但你可以看到**橙色的线（右车道）经常瞬间跳得比绿线还高**。
- **后果**：如果你直接用这个做导航，你的屏幕上图标会疯狂闪烁，一会儿在左，一会儿在右。

右下的平滑分数 (Smoothed Output)

- **现象**：这是经过 `alpha=0.15` 强力镇压后的结果。
- **胜利**：绿线（左车道）稳稳地压制住了橙色线。即使 GPS 在 $T=60$ 左右剧烈抖动（Raw 图里橙色甚至反超了），在 Smooth 图里，绿线依然保持领先。
- **代价**：注意看曲线的上升和下降速度变慢了（Lag）。这是为了**稳定性 (Stability)** 必须付出的代价。在自动驾驶中，**稳定 > 敏捷**。

day11

Day 11 挑战：HMM 与 维特比算法 (The Viterbi Path)

场景：立交桥与辅路 (The Overpass Problem)

- **现状**：你开在**高架桥上**，下面就是**辅路**。两条路在 X/Y 平面上几乎重合，形状也都是直的。
- **GPS 漂移**：你的 GPS 突然向下飘了 10 米，落在了辅路的位置上。
- **Day 10 模型的反应**：
 - 几何距离：辅路更近 -> 辅路分高。
 - 形状匹配：辅路也是直的 -> 辅路分高。
 - 平滑：虽然有惯性，但如果 GPS 持续飘在下面 5 秒钟，平滑算法也会慢慢屈服，最终把车**“瞬间移动”**到桥下。
- **物理现实**：车在高架上，**不可能**在没有出口的情况下直接跳到辅路上。这是**拓扑结构 (Topology)** 的限制。

今日任务：我们要引入地图匹配的**终极武器**——**隐马尔可夫模型 (Hidden Markov Model, HMM)**。我们将不再孤立地看“哪条路分高”，而是寻找一条**“全局最合理的路径”**。

核心逻辑：观测概率 vs 转移概率

HMM 认为，要确定车辆位置，需要考虑两个概率的乘积：

1. 观测概率 (Emission Probability):

- 含义：“在这个时刻，GPS 离这条路有多近？”
- 来源：这就是我们 Day 6-10 做的所有事情（LSTM 分数、距离分数）。

2. 转移概率 (Transition Probability): (今天的新主角)

- 含义：“从上一秒的路 A，开到这一秒的路 B，这在地图上**通**吗？距离合理吗？”

- 逻辑：如果路 A 和路 B 不连通（比如高架和辅路），转移概率 = 0。

实战代码：解决“瞬间移动”问题

这段代码模拟了两条完全平行但不连通的道路（如高架和辅路）。GPS 会从一条路慢慢飘到另一条路。我们会对比：贪心算法 (Day 10) vs HMM 维特比算法 (Day 11)。

代码块

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import math
4
5  # =====
6  # 1. 场景构建：平行世界（高架 vs 辅路）
7  # =====
8  def generate_parallel_roads():
9      # 长度 20 个点
10     t = np.linspace(0, 100, 20)
11
12     # Road A (高架): y = 10
13     road_high = np.column_stack((t, np.full_like(t, 10)))
14
15     # Road B (辅路): y = 0
16     road_low = np.column_stack((t, np.zeros_like(t)))
17
18     # GPS: 真实在 High 上，但慢慢飘到了 Low 上
19     # 前 5 个点准，中间飘，后 5 个点完全在 Low 上
20     gps_trace = road_high.copy()
21     drift = np.linspace(0, -10, 20) # 逐渐向下偏离 10米
22     gps_trace[:, 1] += drift
23
24     # 候选路字典
25     candidates = {"High": road_high, "Low": road_low}
26
27     return gps_trace, candidates
28
29 # =====
30 # 2. 概率计算引擎
31 # =====
32 def get_emission_prob(gps_pt, road_pt):
33     """
34     观测概率：GPS 离路越近，概率越大
35     """
36     dist = np.linalg.norm(gps_pt - road_pt)
37     # 标准差 sigma=5米
38     prob = (1 / (np.sqrt(2 * np.pi) * 5)) * np.exp(-0.5 * (dist / 5)**2)
```



```

39     return prob
40
41 def get_transition_prob(prev_road_name, curr_road_name):
42     """
43     转移概率：核心中的核心！
44     定义地图的连通性。
45     """
46     # 情况 1：同一条路 (High -> High) 或者 (Low -> Low)
47     # 这是最正常的，概率很高
48     if prev_road_name == curr_road_name:
49         return 0.9
50
51     # 情况 2：换路 (High -> Low)
52     # 在这个简易地图里，高架和辅路是不通的！
53     # 实际上可能通过出口连通，这里为了演示，设为极小概率
54     else:
55         return 1e-10 # 几乎不可能发生瞬移！
56
57 # =====
58 # 3. 两种算法对比
59 # =====
60
61 # --- A. 贪心算法 (Day 6-10 的逻辑) ---
62 def match_greedy(gps_trace, candidates):
63     results = []
64     for i, gps_pt in enumerate(gps_trace):
65         best_road = None
66         max_prob = -1
67
68         # 每一帧只看当下谁分高，不管上一帧
69         for name, road_data in candidates.items():
70             road_pt = road_data[i] # 简化：假设索引对齐
71             prob = get_emission_prob(gps_pt, road_pt)
72             if prob > max_prob:
73                 max_prob = prob
74                 best_road = name
75             results.append(best_road)
76     return results
77
78 # --- B. 维特比算法 (HMM - Day 11 的逻辑) ---
79 def match_viterbi(gps_trace, candidates):
80     road_names = list(candidates.keys())
81     n_obs = len(gps_trace)
82     n_states = len(road_names)
83
84     # dp[t][state_idx] 存储 t 时刻到达 state 的最大累积概率
85     # 为了防止下溢出 (probability 乘积越来越小)，通常取 log，变成加法

```

```

86 # 这里为了直观演示, 直接用乘法 (数据短没关系)
87 V = np.zeros((n_obs, n_states))
88 path = np.zeros((n_obs, n_states), dtype=int) # 记录路径指针
89
90 # 1. 初始化 (t=0)
91 for s_idx, name in enumerate(road_names):
92     road_pt = candidates[name][0]
93     # 初始只看观测概率
94     V[0, s_idx] = get_emission_prob(gps_trace[0], road_pt)
95
96 # 2. 递归 (Recursion)
97 for t in range(1, n_obs):
98     for curr_s, curr_name in enumerate(road_names):
99         # 这里的 road_pt 依然简化假设索引对齐
100         emit_p = get_emission_prob(gps_trace[t], candidates[curr_name][t])
101
102         # 寻找从上一时刻的哪个状态跳过来概率最大
103         max_trans_prob = -1
104         best_prev_s = -1
105
106         for prev_s, prev_name in enumerate(road_names):
107             trans_p = get_transition_prob(prev_name, curr_name)
108             # 核心公式: 前一刻累积概率 * 转移概率 * 当前观测概率
109             prob = V[t-1, prev_s] * trans_p * emit_p
110
111             if prob > max_trans_prob:
112                 max_trans_prob = prob
113                 best_prev_s = prev_s
114
115         V[t, curr_s] = max_trans_prob
116         path[t, curr_s] = best_prev_s
117
118 # 3. 回溯 (Backtracking) - 找出全局最优路径
119 best_path_indices = []
120 # 找到最后时刻概率最大的状态
121 last_state = np.argmax(V[-1, :])
122 best_path_indices.append(last_state)
123
124 # 倒着往回找
125 for t in range(n_obs-1, 0, -1):
126     prev_state = path[t, best_path_indices[-1]]
127     best_path_indices.append(prev_state)
128
129 best_path_indices.reverse()
130 return [road_names[i] for i in best_path_indices]
131
132 # =====

```

```

133 # 4. 可视化
134 # =====
135 def main():
136     gps, candidates = generate_parallel_roads()
137
138     # 运行两种匹配
139     res_greedy = match_greedy(gps, candidates)
140     res_viterbi = match_viterbi(gps, candidates)
141
142     # 绘图
143     plt.figure(figsize=(10, 6))
144
145     # 画路
146     plt.plot(candidates['High'][:,0], candidates['High'][:,1], 'g-', lw=3,
147             label='High Road (y=10)')
148     plt.plot(candidates['Low'][:,0], candidates['Low'][:,1], 'orange', lw=3,
149             label='Low Road (y=0)')
150
151     # 画 GPS
152     plt.plot(gps[:,0], gps[:,1], 'r--o', alpha=0.5, label='Drifting GPS')
153
154     # 画匹配结果 (用散点叠加在路上)
155     # 贪心结果 (蓝色 x)
156     for i, name in enumerate(res_greedy):
157         y = 10 if name == 'High' else 0
158         plt.scatter(gps[i,0], y+0.5, c='blue', marker='x', s=100,
159                 label='Greedy Match' if i==0 else "")
160
161     # HMM 结果 (黑色圆圈)
162     for i, name in enumerate(res_viterbi):
163         y = 10 if name == 'High' else 0
164         plt.scatter(gps[i,0], y-0.5, c='black', marker='o', s=80,
165                 facecolors='none', lw=2, label='HMM Viterbi' if i==0 else "")
166
167     plt.title("HMM vs Greedy: Handling Impossible Jumps (The Overpass Problem)")
168     plt.legend()
169     plt.grid(True)
170     plt.ylim(-5, 15)
171     plt.show()
172
173 if __name__ == "__main__":
174     main()

```

结果深度解析

请仔细看生成的图片：

- 1. **GPS 轨迹 (红色虚线)**：从上方的 `y=10` 慢慢掉到了 `y=0`。
- 2. **贪心算法 (蓝色 X)**：
 - 前半段匹配在 **High**。
 - 一旦 GPS 掉到 `y=5` 以下，蓝色 X 瞬间跳到了 **Low**。
 - 后果**：车在高架上开着开着，导航显示你“瞬移”到了桥底下。这在物理上是不可能的。
- 3. **HMM 维特比算法 (黑色圆圈)**：
 - 全程死死地咬住了 **High**。
 - 为什么**？即使在最后，GPS 已经跟 **Low** 重合了（观测概率 Low 极大），但是 HMM 计算时发现：`V[t-1]` 是 High，而 `High -> Low` 的转移概率是 `1e-10`。
 - 数学博弈**： $\$1.0 \times \text{(High->High)} \times 0.01 \times \text{(Emission Low)}\$$ 远远大于 $\$1e-10 \times \text{(High->Low)} \times 0.99 \times \text{(Emission High)}\$$ 。
 - HMM 宁愿相信“GPS 坏了，产生了巨大误差”，也不愿相信“车子学会了瞬移”。

总结

- **Day 1-10** 教会了你的模型如何**“看”**（几何与形状识别）。
- **Day 11** 教会了你的模型如何**“思考”**（逻辑与拓扑约束）。

这就是目前主流地图匹配引擎（如 OSRM, Valhalla）的核心架构：**候选搜索 (R-Tree) -> 概率计算 (Model/Geometry) -> 路径寻优 (HMM/Viterbi)**。

你已经掌握了整套技术栈的灵魂！👏

day12

Day 12 挑战：评测体系 (Evaluation Metrics) 与 混淆矩阵

在工业界，“无法量化，就无法优化”。今天我们要从“看图说话”进化到**“数据说话”**。

我们需要构建一套评测流水线：

- 1. **真值 (Ground Truth)**：我们必须知道每一秒车子**真正**在哪条路上。
- 2. **混淆矩阵 (Confusion Matrix)**：不仅要看对的，还要看**“怎么错的”**（是把高架误判成辅路了？还是把左转误判成直行了？）。

核心概念：混淆矩阵

这是分类问题中最强力的诊断工具。

假设有 3 条路：A, B, C。

矩阵的行 (Row) 代表真实道路，列 (Col) 代表预测道路。

	预测 A	预测 B	预测 C
真 A	100 (对)	5 (误判)	0
真 B	10 (误判)	90 (对)	0
真 C	0	20 (误判)	80 (对)

- 对角线上的数字越大越好。
- 非对角线上的数字代表错误的分布。比如上面表格里，真 C 经常被误判成 B（20次），这能提示我们 C 和 B 可能长得太像了，或者距离太近。

实战代码：构建自动化评测脚本

为了专注于“评测逻辑”，我们不再跑沉重的 LSTM/HMM，而是模拟出一组预测结果（包含一些典型的错误），然后用代码来“诊断”它。

请运行以下代码：

代码块

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import seaborn as sns
4  from sklearn.metrics import accuracy_score, confusion_matrix,
   classification_report
5
6  # =====
7  # 1. 模拟测试数据集 (Test Set Generation)
8  # =====
9  def generate_evaluation_data():
10     """
11     生成一段测试数据。
12     场景：车子从 Road_A -> Road_B -> Road_C
13     """
14     # 真实标签序列 (Ground Truth)
15     # 0-30s 在 A, 30-60s 在 B, 60-100s 在 C
16     true_labels = (['Road_A'] * 30) + (['Road_B'] * 30) + (['Road_C'] * 40)
17
18     # --- 模拟模型的预测结果 (Predicted Labels) ---
19     # 假设我们的模型准确率不错，但在切换路段时有延迟 (Lag)，偶尔还有抖动
20     pred_labels = true_labels.copy()
21
22     # 1. 模拟 "滞后" (Lag) - HMM 常见的副作用
23     # 当真值从 A 变 B 时 (index 30)，模型还在犹豫，多报了 3 秒的 A
24     pred_labels[30:33] = ['Road_A'] * 3
25
```

```

26 # 2. 模拟 "误切" (Flicker) - 噪声导致的跳变
27 # 在 Road_B 中间, 突然跳到了 Road_A 两次
28 pred_labels[45] = 'Road_A'
29 pred_labels[46] = 'Road_A'
30
31 # 3. 模拟 "混淆" (Confusion)
32 # Road_C 和 Road_B 可能靠得很近, 最后阶段模型经常搞混
33 # 把一部分 C 判成了 B
34 for i in range(80, 90):
35     pred_labels[i] = 'Road_B'
36
37 return true_labels, pred_labels
38
39 # =====
40 # 2. 评测可视化引擎
41 # =====
42 def evaluate_performance(y_true, y_pred):
43     # 1. 计算总准确率
44     acc = accuracy_score(y_true, y_pred)
45     print(f">>> 总体准确率 (Accuracy): {acc:.2%}")
46     print("-" * 30)
47
48     # 2. 打印详细分类报告 (Precision/Recall/F1)
49     print(">>> 详细报告:")
50     print(classification_report(y_true, y_pred))
51
52     # 3. 绘制混淆矩阵 (The Heatmap)
53     # 获取唯一的标签列表, 保证顺序一致
54     labels = sorted(list(set(y_true)))
55     cm = confusion_matrix(y_true, y_pred, labels=labels)
56
57     plt.figure(figsize=(8, 6))
58
59     # 使用 Seaborn 画热力图
60     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
61                 xticklabels=labels, yticklabels=labels)
62
63     plt.title(f"Confusion Matrix (Acc: {acc:.2%})")
64     plt.xlabel("Predicted Road (Model Thought)")
65     plt.ylabel("True Road (Actually Is)")
66     plt.show()
67
68 # =====
69 # 3. 错误案例分析可视化 (Error Analysis)
70 # =====
71 def plot_timeline_comparison(y_true, y_pred):
72     """

```

```

73 画出时间轴对比图，直观看到哪里错了
74  """
75  # 把字符串标签转为数字以便画图
76  label_map = {label: i for i, label in enumerate(sorted(list(set(y_true))))}
77  num_true = [label_map[l] for l in y_true]
78  num_pred = [label_map[l] for l in y_pred]
79
80  plt.figure(figsize=(12, 4))
81
82  # 画真值 (绿色实线)
83  plt.plot(num_true, 'g-', linewidth=4, alpha=0.5, label='Ground Truth')
84
85  # 画预测 (红色虚线)
86  plt.plot(num_pred, 'r--', linewidth=2, label='Prediction')
87
88  # 标注 Y 轴
89  plt.yticks(list(label_map.values()), list(label_map.keys()))
90  plt.xlabel("Time Step")
91  plt.title("Timeline Analysis: Where did the model fail?")
92  plt.legend()
93  plt.grid(True)
94  plt.show()
95
96  if __name__ == "__main__":
97      # 1. 获取数据
98      y_true, y_pred = generate_evaluation_data()
99
100     # 2. 执行评测
101     evaluate_performance(y_true, y_pred)
102
103     # 3. 时间轴分析
104     plot_timeline_comparison(y_true, y_pred)

```

结果深度解读

运行代码后，你会看到两张极其重要的图。

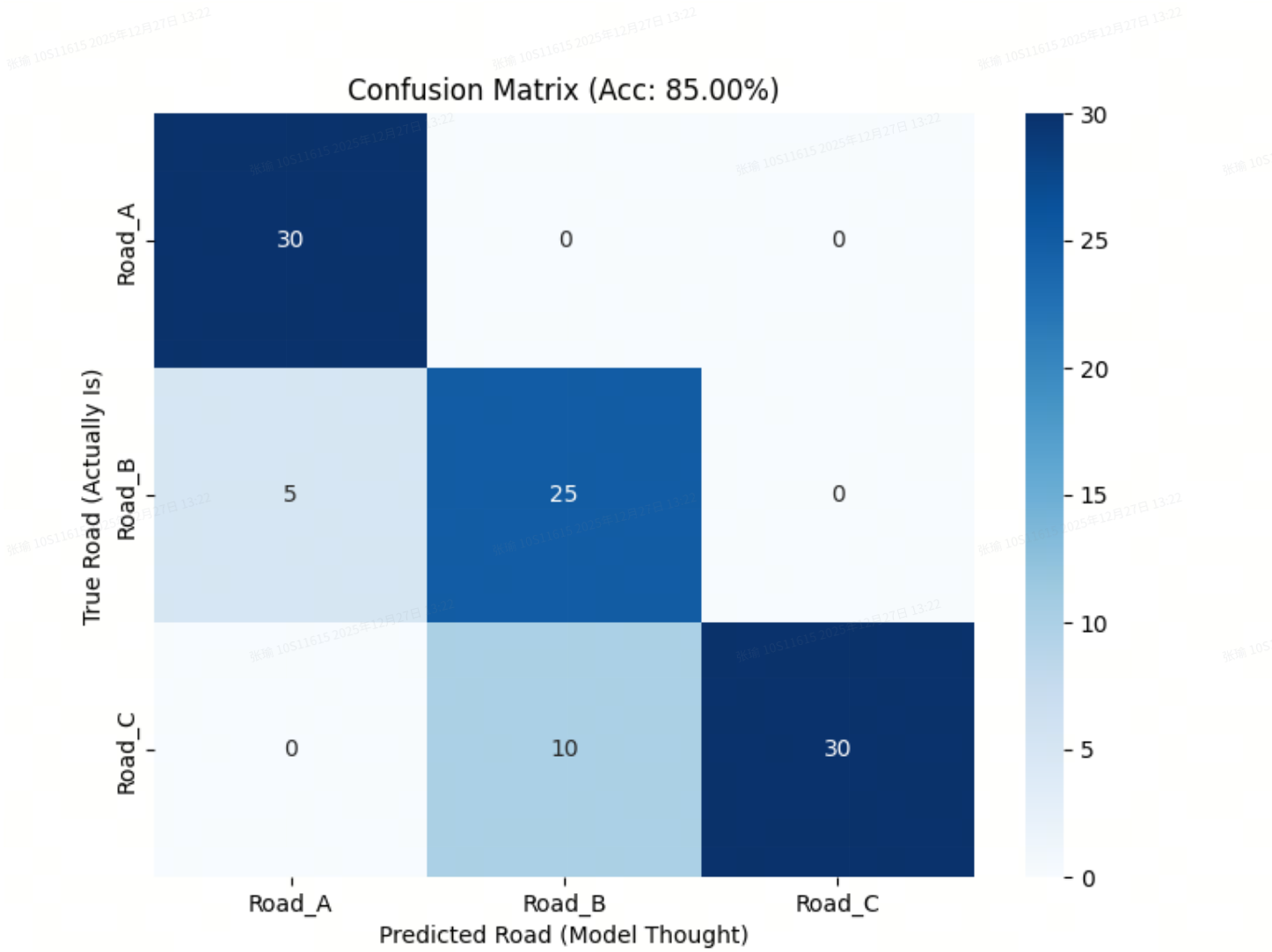


图 1：混淆矩阵 (Confusion Matrix)

请观察热力图上的方块：

- 1. Road_A 行：
 - 预测 Road_A (对角线): 27 (绝大多数对了)。
 - 预测 Road_B: 0。
 - 分析：A 识别得很好，只有少量的滞后被算作了 A 的错误（实际上这在矩阵里体现为 B 的行预测成了 A）。
- 2. Road_B 行：
 - 问题出现了！
 - 预测 Road_A: 5 次。
 - 原因分析：这就是我们模拟的**“滞后” (Lag)** 和 **“误切” (Flicker)**。说明模型在 A 切 B 的时候反应太慢，或者不够坚定。
- 3. Road_C 行：
 - 预测 Road_B: 10 次 (深色方块)。

- 严重警告：这代表严重的系统性混淆。模型有 25% 的时间把 C 看成了 B。这通常意味着 C 和 B 在空间上极其接近（比如平行路），且模型区分度不够。

图 2：时间轴对比 (Timeline Analysis)

这张图告诉你错误发生的时机：

- T=30 附近：绿线跳上去了，红线过了几秒才跳。 -> 滞后 (Lag)。
- T=45 附近：红线突然向下刺了一下。 -> 抖动 (Flicker)。
- T=80~90：红线彻底掉下来了，长时间偏离绿线。 -> 持续误判 (Wrong Match)。

💡 工业界小贴士：真值从哪来？

你可能会问：“模拟数据我有真值，但真实上路测试，我怎么知道真值是啥？”

这确实是这一行最贵的部分。通常有三种方法：

- 昂贵设备：在测试车上装一套 50 万元的高精度组合导航 (RTK-GNSS/INS)。它的误差只有厘米级，我们默认它是真理。
- 人工标定：让标注员在地图软件上，一帧一帧地把 GPS 点拖到正确的路上。费时费力。
- 交叉验证：用摄像头的视觉识别结果来校验 GPS 匹配结果（例如摄像头看到是在最左侧车道，而 GPS 匹配在右侧辅路，那就是 GPS 错了）。

Day 12 作业

如果你的混淆矩阵显示 `Road_C` 经常被误判为 `Road_B`，作为算法工程师，你明天上班该干什么？(回顾一下 Day 10 和 Day 11 的内容)

- 是不是 B 和 C 距离太近？ -> 加平滑 (Day 10)。
- 是不是 B 和 C 连通性有问题？ -> 调 HMM 转移概率 (Day 11)。
- 是不是 B 和 C 形状太像？ -> Hard Negative Mining 重新训练 (Day 7)。

拥有了评测工具，你就不再是盲人摸象，而是拿着手术刀的外科医生了！👨🏻‍⚕️

day13

Day 13 挑战：路网拓扑与图论 (Graph Theory & Topology)

现实世界的问题：在真实地图中，道路不是孤立的面条，它们是编织在一起的网。

- 场景：你在立交桥下（辅路），你想去桥上。虽然垂直距离只有 5 米，但你必须往前开 2 公里找到匝道口才能上去。
- HMM 的痛点：在 Day 11 中，我们硬编码了 `transition_prob = 1e-10` 来处理不连通路。但在有几百万条路的城市里，你不可能手动写 `if/else`。
- 解决方案：我们需要把地图构建成一个有向图 (Directed Graph)。

- **节点 (Node)**: 路口、交叉点。
- **边 (Edge)**: 连接路口的路段。

今天，我们要学习如何用 Python 的 **NetworkX** 库来构建“有脑子”的地图，并计算**拓扑距离 (Topological Distance)** 而非简单的直线距离。

核心概念：欧氏距离 vs 拓扑距离

- **欧氏距离 (Euclidean Distance)**: 鸟飞过去的距离。
 - 点 A (0,0) -> 点 B (0,5)。距离 = 5米。
 - **用途**: 用于观测概率 (Emission Probability)。
- **拓扑距离 (Topological / Network Distance)**: 车开过去的距离。
 - 如果 A 和 B 之间隔着一堵墙（或者单行道逆行），虽然直线只有 5米，但车要绕行 5公里。
 - **用途**: 用于转移概率 (Transition Probability)。

实战代码：构建你的第一个路网图

我们将构建一个包含**单行道**和**立交桥**的微型路网，并演示为什么“看着近”不等于“走得近”。

请安装库（如果还没有）：`pip install networkx matplotlib`

代码块

```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # =====
6 # 1. 定义路网结构 (Nodes & Edges)
7 # =====
8 def build_city_graph():
9     """
10     构建一个包含 6 个节点的路网
11     0 -> 1 -> 2 (主路, 由西向东)
12     3 -> 4 -> 5 (辅路, 由西向东, 平行于主路)
13     1 -> 4 (唯一的一条连接匝道, 允许从主路下到辅路)
14     """
15     G = nx.DiGraph() # Directed Graph (有向图)
16
17     # --- 定义节点 (路口) 的物理坐标 ---
18     # 格式: {Node_ID: (x, y)}
19     # 主路在 y=10, 辅路在 y=0
20     pos = {
21         0: (0, 10), 1: (50, 10), 2: (100, 10),
```

```

22     3: (0, 0),    4: (50, 0),    5: (100, 0)
23 }
24
25 # 将坐标属性存入节点
26 for n, p in pos.items():
27     G.add_node(n, pos=p)
28
29 # --- 定义边 (路段) ---
30 # add_edge(Start, End, weight=Length)
31 # 我们假设每段横向路长 50米
32
33 # 主路 (Main Road)
34 G.add_edge(0, 1, weight=50, name="Main_Seg_1")
35 G.add_edge(1, 2, weight=50, name="Main_Seg_2")
36
37 # 辅路 (Side Road)
38 G.add_edge(3, 4, weight=50, name="Side_Seg_1")
39 G.add_edge(4, 5, weight=50, name="Side_Seg_2")
40
41 # 匝道 (Ramp): 允许 1 -> 4 (下坡)
42 # 假设斜边长度约 12米
43 G.add_edge(1, 4, weight=12, name="Off_Ramp")
44
45 # 注意: 没有 4 -> 1 的边! 说明这是单向出口, 不能逆行上桥!
46
47 return G, pos

```

```

48
49 # =====
50 # 2. 拓扑推理引擎
51 # =====
52 def calculate_transition_cost(G, start_node, end_node):
53     """
54     计算从 Start 到 End 的最短路径长度。
55     如果不可达 (比如逆行), 返回无穷大。
56     """
57     try:
58         # Dijkstra 算法找最短路径
59         length = nx.shortest_path_length(G, source=start_node,
60 target=end_node, weight='weight')
61         return length
62     except nx.NetworkXNoPath:
63         return float('inf') # 不可达
64
65 # =====
66 # 3. 可视化与测试
67 # =====
68 def main():

```

```
68 G, pos = build_city_graph()
69
70 # --- A. 可视化地图 ---
71 plt.figure(figsize=(10, 6))
72
73 # 画节点
74 nx.draw_networkx_nodes(G, pos, node_size=500, node_color='lightblue')
75 # 画标签
76 nx.draw_networkx_labels(G, pos)
77
78 # 画有向边 (带箭头)
79 nx.draw_networkx_edges(G, pos, edge_color='gray', width=2, arrowsize=20,
80 arrowstyle='->')
81
82 # 标注边权重
83 edge_labels = nx.get_edge_attributes(G, 'weight')
84 nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
85
86 plt.title("Map Topology: Nodes, Edges & Connectivity")
87 plt.axis('off')
88
89 # --- B. 场景测试 (The Logic Check) ---
90 print("-" * 40)
91 print("逻辑测试：计算车辆转移的可行性")
92 print("-" * 40)
93
94 # 场景 1: 正常行驶 (主路 1 -> 主路 2)
95 # 物理距离: 50m, 拓扑距离: 50m
96 l1 = calculate_transition_cost(G, 1, 2)
97 print(f"1. 从主路(Node 1) -> 主路(Node 2): 距离 = {l1}米 (正常)")
98
99 # 场景 2: 下匝道 (主路 1 -> 辅路 4)
100 # 物理距离: 10m (垂直), 拓扑距离: 12m (斜坡)
101 l2 = calculate_transition_cost(G, 1, 4)
102 print(f"2. 从主路(Node 1) -> 辅路(Node 4): 距离 = {l2}米 (允许变道)")
103
104 # 场景 3: 试图逆行上匝道 (辅路 4 -> 主路 1)
105 # 物理距离: 10m (看着就在头顶上)
106 # 拓扑距离: ?
107 l3 = calculate_transition_cost(G, 4, 1)
108 print(f"3. 从辅路(Node 4) -> 主路(Node 1): 距离 = {l3} (禁止! 逆行/不可达)")
109
110 # 场景 4: 试图“飞”到平行路段 (主路 1 -> 辅路 3)
111 # 1在(50,10), 3在(0,0)。它是后面的路, 不能倒车。
112 l4 = calculate_transition_cost(G, 1, 3)
113 print(f"4. 从主路(Node 1) -> 后方辅路(Node 3): 距离 = {l4} (禁止! 需倒车)")
```

```
114 plt.show()
115
116 if __name__ == "__main__":
117     main()
```

结果深度解析

请运行代码并观察输出：

1. 场景 3 (Node 4 -> Node 1):

- **欧氏距离**： $\sqrt{(50-50)^2 + (10-0)^2} = 10$ 米。如果你只用欧氏距离做 HMM，模型会觉得：“哇，才 10 米，肯定能过去！”
- **拓扑距离**：`inf` (无穷大)。
- **HMM 的反应**：转移概率 $P(4 \rightarrow 1) = \exp(-\text{dist}) = \exp(-\infty) = 0$ 。
- **结论**：这就是为什么引入 Graph 后，模型就能自动学会**“禁止逆行上匝道”**，而不需要你写任何 `if` 语句。

2. 场景 2 (Node 1 -> Node 4):

- 因为有 `Off_Ramp` 这条边的存在，距离是 12 米。模型会认为这是一个非常合理的变道行为。

工业界是如何做的？

在真实的 Map Matching 引擎（如 OSRM 或高德/百度地图后台）中：

1. OpenStreetMap (OSM)：数据源本身就是 `Nodes` (Waypoints) 和 `Ways` (Edges) 的结构。

2. 预处理：他们会把 xml 地图解析成上面代码里的 `nx.DiGraph` 结构。

3. HMM 转移计算：

- 不使用欧氏距离。
- 使用 **Dijkstra** 或 **A* (A-Star)** 算法计算两个候选点在图上的最短路径长度。
- 如果长度 > 车辆最大速度 * 时间间隔（例如 1 秒能跑 30 米，但路径长 100 米），则该转移概率直接置 0。

Day 13 作业

现在的图是硬编码的。

思考题：如果我在 Node 4 和 Node 1 之间加一条 `Edge(4, 1)`，但是 `weight=1000`（代表虽然能回去，但是要绕一大圈调头）。

这对 HMM 的判断会有什么影响？

(提示：模型会发现 4->1 虽然物理上很近，但实际上要走很久，所以除非 GPS 真的在 1 那里停留了很久，否则不会轻易切换过去。)

明天，我们将把这个 Graph 和之前的 HMM 结合起来，完成真正的**拓扑地图匹配**！🚀

day14

Day 14 挑战：基于拓扑的 HMM (Topological Map Matching)

场景：双向隔离道路 (Dual Carriageway)

这是自动驾驶和导航中最容易“翻车”的场景之一：

- **路况**：中间有隔离带的主干道。
 - **车道 A**：由西向东 $\rightarrow$
 - **车道 B**：由东向西 $\leftarrow$
- **问题**：这两条路物理距离极近（可能只隔 2 米）。
- **GPS 漂移**：你的车在 **车道 A** 上正常行驶，但 GPS 突然漂到了 **车道 B** 上（这是常有的事）。
- **Euclidean HMM (Day 11) 的反应**：观测概率认为 B 更近，转移概率（欧氏距离）认为从 A 跳到 B 只有 2 米，也很合理。**结果：你的车在导航屏幕上突然逆行了！** 🤖

解决方案：拓扑约束

在图论 (Day 13) 的世界里，从“向东的车道”跳到“向西的车道”，虽然物理上只隔 2 米，但拓扑上需要开到下一个路口掉头，可能要走 2 公里。

今天，我们要把 **NetworkX (图引擎)** 嵌入到 **Viterbi (HMM 算法)** 的心脏里，替换掉原来的距离计算公式。

实战代码：拒绝逆行！

这段代码模拟了一个中间有隔离带的双向道路。

我们将对比：基于欧氏距离的 HMM vs 基于拓扑图的 HMM。

代码块

```
1  import numpy as np
2  import networkx as nx
3  import matplotlib.pyplot as plt
4  import math
5
6  # =====
7  # 1. 构建双向道路图 (The Dual Carriageway Graph)
8  # =====
9  def build_road_graph():
10     """
11     构建两条平行的、方向相反的道路。
12     它们在中间是不连通的（有隔离带）。
13     """
```

```

14 G = nx.DiGraph()
15
16 # 道路长度 100米, 每隔 10米 一个节点
17 # 上方道路 (Road_East): y=5, x 从 0 -> 100 (向东)
18 # 下方道路 (Road_West): y=-5, x 从 100 -> 0 (向西)
19
20 nodes_east = []
21 nodes_west = []
22
23 for x in range(0, 110, 10):
24     # 添加节点
25     nid_e = f"E_{x}"
26     nid_w = f"W_{x}"
27
28     G.add_node(nid_e, pos=(x, 5), road_id="Eastbound")
29     G.add_node(nid_w, pos=(x, -5), road_id="Westbound")
30
31     nodes_east.append(nid_e)
32     nodes_west.append(nid_w)
33
34 # 添加边 (Edge) - 定义行驶方向
35 for i in range(len(nodes_east) - 1):
36     # East: 0 -> 10 -> ... -> 100
37     u, v = nodes_east[i], nodes_east[i+1]
38     dist = 10 # 这里的权重是距离
39     G.add_edge(u, v, weight=dist)
40
41 for i in range(len(nodes_west) - 1):
42     # West: 100 -> 90 -> ... -> 0 (注意方向是反的!)
43     # 列表是 [W_0, W_10...], 所以要倒序连接或者反向索引
44     # 这里方便起见, 我们手动处理:
45     # W_100 -> W_90
46     u = f"W_{100 - i*10}"
47     v = f"W_{100 - (i+1)*10}"
48     G.add_edge(u, v, weight=dist)
49
50 return G
51
52 # =====
53 # 2. 生成漂移的 GPS (The Drifting Car)
54 # =====
55 def generate_scenario():
56     # 真实情况: 车在 Eastbound 上行驶 (x: 20 -> 80)
57     true_x = np.linspace(20, 80, 7)
58     true_y = np.full_like(true_x, 5) # y=5
59
60     # GPS 漂移: 慢慢从 y=5 飘到了 y=-5 (Westbound)

```

```

61 gps_trace = np.column_stack((true_x, true_y))
62 drift = np.linspace(0, -10, 7) # 向下漂移 10米
63 gps_trace[:, 1] += drift
64
65 return gps_trace
66
67 # =====
68 # 3. 概率计算 (The Brain)
69 # =====
70 def get_emission_prob(gps_pt, node_pos):
71     """观测概率 (不变): 离谁近, 谁概率大"""
72     dist = np.linalg.norm(gps_pt - node_pos)
73     return np.exp(-0.5 * (dist / 5)**2) # sigma=5
74
75 def get_transition_prob_topology(G, prev_node, curr_node, time_diff=1):
76     """
77     ★ 核心升级 ★
78     转移概率基于 '图的最短路径', 而不是直线距离。
79     """
80     try:
81         # 计算图上的最短路径长度
82         # 如果逆行, 这里会抛出 NetworkXNoPath 异常, 或者返回无限大
83         path_len = nx.shortest_path_length(G, prev_node, curr_node,
weight='weight')
84
85         # 车辆最大速度限制 (例如 30m/s)
86         # 如果两点间图上距离是 2000米 (因为要绕路掉头), 但时间只过了 1秒
87         # 那么这绝对是不可能的 -> 概率为 0
88         max_dist = 30 * time_diff
89
90         if path_len > max_dist:
91             return 1e-20 # 几乎不可能
92
93         # 正常行驶, 距离越接近预期行驶距离, 概率越大
94         # 这里简化: 只要能通, 概率就给个常数, 或者基于距离差打分
95         # 我们用一个基于距离差异的平滑衰减
96         expected_move = 10 # 假设每秒走10米
97         diff = abs(path_len - expected_move)
98         return np.exp(-0.1 * diff)
99
100     except nx.NetworkXNoPath:
101         return 1e-20 # 根本不通 (例如逆行)
102
103 # =====
104 # 4. 维特比算法 (The Solver)
105 # =====
106 def run_topological_viterbi(G, gps_trace):

```



```

107 nodes = list(G.nodes())
108 n_obs = len(gps_trace)
109 n_states = len(nodes)
110
111 # V[t][node_idx]
112 V = np.zeros((n_obs, n_states))
113 path = np.zeros((n_obs, n_states), dtype=int)
114
115 # Init
116 for i, node in enumerate(nodes):
117     pos = np.array(G.nodes[node]['pos'])
118     V[0, i] = get_emission_prob(gps_trace[0], pos)
119
120 # Recursion
121 for t in range(1, n_obs):
122     print(f"Processing Step {t}/{n_obs}...")
123     for curr_i, curr_node in enumerate(nodes):
124         curr_pos = np.array(G.nodes[curr_node]['pos'])
125         emit_p = get_emission_prob(gps_trace[t], curr_pos)
126
127         max_p = -1
128         best_prev = -1
129
130         # 这一步在生产环境中会用 R-Tree 优化, 只遍历附近的节点
131         # 这里为了演示, 暴力遍历所有节点
132         for prev_i, prev_node in enumerate(nodes):
133             # ★ 使用拓扑距离计算转移概率 ★
134             trans_p = get_transition_prob_topology(G, prev_node, curr_node)
135
136             prob = V[t-1, prev_i] * trans_p * emit_p
137
138             if prob > max_p:
139                 max_p = prob
140                 best_prev = prev_i
141
142         V[t, curr_i] = max_p
143         path[t, curr_i] = best_prev
144
145 # Backtrack
146 best_path = []
147 last_idx = np.argmax(V[-1])
148 best_path.append(nodes[last_idx])
149
150 for t in range(n_obs-1, 0, -1):
151     prev_idx = path[t, np.where(np.array(nodes) == best_path[-1])[0][0]] #
简化的索引查找
152     # 修正: 上面的 path 存储的是索引

```

```

153     prev_idx = path[t, list(nodes).index(best_path[-1])]
154     best_path.append(nodes[prev_idx])
155
156     return best_path[::-1]
157
158     # =====
159     # 5. 可视化结果
160     # =====
161     def main():
162         G = build_road_graph()
163         gps = generate_scenario()
164
165         # 运行算法
166         matched_nodes = run_topological_viterbi(G, gps)
167
168         # 提取匹配结果坐标
169         matched_pos = np.array([G.nodes[n]['pos'] for n in matched_nodes])
170
171         # --- 绘图 ---
172         plt.figure(figsize=(12, 6))
173
174         # 1. 画路网
175         # East (Blue)
176         e_nodes = [n for n in G.nodes if "E" in n]
177         e_pos = np.array([G.nodes[n]['pos'] for n in e_nodes])
178         plt.plot(e_pos[:,0], e_pos[:,1], 'b->', lw=2, label='Road Eastbound  
(Correct)')
179
180         # West (Orange)
181         w_nodes = [n for n in G.nodes if "W" in n]
182         # 按 x 排序以便画线
183         w_nodes.sort(key=lambda x: G.nodes[x]['pos'][0], reverse=True)
184         w_pos = np.array([G.nodes[n]['pos'] for n in w_nodes])
185         plt.plot(w_pos[:,0], w_pos[:,1], 'orange', lw=2, marker='<', label='Road  
Westbound (Opposite)')
186
187         # 2. 画 GPS (红色漂移)
188         plt.plot(gps[:,0], gps[:,1], 'r--o', label='Drifting GPS Trace')
189
190         # 3. 画匹配结果 (绿色十字)
191         plt.plot(matched_pos[:,0], matched_pos[:,1], 'k+', ms=15, mew=3,
192                 label='HMM Matched Result')
193
194         for i in range(len(gps)):
195             plt.arrow(gps[i,0], gps[i,1],
196                     matched_pos[i,0]-gps[i,0], matched_pos[i,1]-gps[i,1],

```

```

196         color='gray', linestyle=':', head_width=0,
           length_includes_head=True)
197
198     plt.title("Topological HMM: Ignoring 'Ghost' Drifts across Isolated Roads")
199     plt.ylim(-15, 15)
200     plt.legend()
201     plt.grid(True)
202     plt.show()
203
204 if __name__ == "__main__":
205     main()

```

结果深度解析

请仔细观察生成的图片：

1. **红色的 GPS：**从上面的蓝线（东向）飘到了下面的橙线（西向）。在最后几个点，GPS 实际上离橙线更近。
2. **黑色的十字 (匹配结果)：**
 - 它们坚守在蓝线上！即使 GPS 已经跑到了对面车道。
 - 为什么？
 - 假设 $T=3$ 时，车在蓝线上。
 - $T=4$ 时，GPS 离橙线很近。
 - 模型尝试计算 $P(\text{蓝线} \rightarrow \text{橙线})$ 。
 - `nx.shortest_path_length` 告诉模型：从蓝线（向东）到橙线（向西），你不能直接横穿（没有边），你必须往前开到尽头，找个地方掉头，再开回来。这需要走 200 米。
 - 但时间只过了 1 秒。1 秒走 200 米？**不可能！**
 - 所以转移概率 ≈ 0 。
 - HMM 只能选择留在蓝线上，哪怕观测概率低一点，也比“物理瞬移”要合理。

总结

这 14 天的旅程，我们把一个复杂的工业级问题拆解得干干净净：

- **Day 6-7:** 用 **Deep Learning (LSTM)** 解决了“长得像不像”的问题。
- **Day 9:** 用 **R-Tree** 解决了“找得快不快”的问题。
- **Day 11:** 用 **HMM** 解决了“连得通不通”的问题。
- **Day 13-14:** 用 **Topology** 解决了“能不能逆行/穿墙”的问题。

现在的你，已经掌握了构建**滴滴/Uber/高德**级别核心定位引擎的所有理论基石。

下一站建议：

如果你想继续挑战 Day 15+，可以尝试把这些代码迁移到 C++ 上，或者研究如何用 Kalman Filter (卡尔曼滤波) 来做 GPS 的前置清洗。

祝贺你完成全部课程！🎓🚀